3.15 Write a function called sum-squares which uses mapcar to find the sum of the squares of a list of integers. The function takes a single list as its argument. Write a lambda function which mapcar uses to find the square of the integers in the list. For example,

```
(sum-squares (2 3 1 4)) should return 30.
```

3.16 Write a PROLOG program that answers questions about family members and relationships. Include predicates and rules which define sister, brother, father, mother, grandchild, grandfather, and uncle. The program should be able to answer queries such as the following:

```
?- father(X, bob).
?- grandson(X, Y).
?- uncle(bill, sue).
?- mother(mary, X).
```

3.17 Trace the search sequence PROLOG follows in satisfying the following goal:

```
?- member(c,[a,b,c,d]).
```

3.18 Write a function called match that takes two arguments, a pattern and a clause. If the first argument, the pattern, is a variable identified by a question mark followed by a lowercase letter (like ?x or ?y), the function should return a list giving the variable and the corresponding clause, since a variable matches anything. If the pattern is not a variable, it should return *t* only if the pattern and the clause are identical. Otherwise, the function should return nil.

PART 2
Knowledge Representation

4
Formalized Symbolic Logics

Starting with this chapter, we begin a study of some basic tools and methodologies used in AI and the design of knowledge-based systems. The first representation scheme we examine is one of the oldest and most important, First Order Predicate Logic (FOPL). It was developed by logicians as a means for formal reasoning, primarily in the areas of mathematics. Following our study of FOPL, we then investigate five additional representation methods which have become popular over the past twenty years. Such methods were developed by researchers in AI or related fields for use in representing different kinds of knowledge.

After completing these chapters, we should be in a position to best choose which representation methods to use for a given application, to see how automated reasoning can be programmed, and to appreciate how the essential parts of a system fit together.

4.1 INTRODUCTION

The use of symbolic logic to represent knowledge is not new in that it predates the modern computer by a number of decades. Even so, the application of logic as a practical means of representing and manipulating knowledge in a computer was not demonstrated until the early 1960s (Gilmore, 1960). Since that time, numerous

systems have been implemented with varying degrees of success. Today, First Order Predicate Logic (FOPL) or Predicate Calculus as it is sometimes called, has assumed one of the most important roles in AI for the representation of knowledge.

A familiarity with FOPL is important to the student of AI for several reasons. First, logic offers the only formal approach to reasoning that has a sound theoretical foundation. This is especially important in our attempts to mechanize or automate the reasoning process in that inferences should be correct and logically sound. Second, the structure of FOPL is flexible enough to permit the accurate representation of natural language reasonably well. This too is important in AI systems since most knowledge must originate with and be consumed by humans. To be effective, transformations between natural language and any representation scheme must be natural and easy. Finally, FOPL is widely accepted by workers in the AI field as one of the most useful representation methods. It is commonly used in program designs and widely discussed in the literature. To understand many of the AI articles and research papers requires a comprehensive knowledge of FOPL as well as some related logics.

Logic is a formal method for reasoning. Many concepts which can be verbalized can be translated into symbolic representations which closely approximate the meaning of these concepts. These symbolic structures can then be manipulated in programs to deduce various facts, to carry out a form of automated reasoning.

In FOPL, statements from a natural language like English are translated into symbolic structures comprised of predicates, functions, variables, constants, quantifiers, and logical connectives. The symbols form the basic building blocks for the knowledge, and their combination into valid structures is accomplished using the syntax (rules of combination) for FOPL. Once structures have been created to represent basic facts or procedures or other types of knowledge, inference rules may then be applied to compare, combine and transform these "assumed" structures into new "deduced" structures. This is how automated reasoning or inferencing is performed.

As a simple example of the use of logic, the statement "All employees of the AI-Software Company are programmers" might be written in FOPL as

$$(\forall x)\ (\text{AI-SOFTWARE-CO-EMPLOYEE}(x) \rightarrow \text{PROGRAMMER}(x))$$

Here, $\forall x$ is read as "for all x" and $\rightarrow$ is read as "implies" or "then." The predicates AI-SOFTWARE-CO-EMPLOYEE(x), and PROGRAMMER(x) are read as "if x is an AI Software Company employee," and "x is a programmer" respectively. The symbol x is a variable which can assume a person's name.

If it is also known that Jim is an employee of AI Software Company,

$$\text{AI-SOFTWARE-CO-EMPLOYEE}(\text{jim})$$

one can draw the conclusion that Jim is a programmer.

$$\text{PROGRAMMER}(\text{jim})$$

The above suggests how knowledge in the form of English sentences can be translated into FOPL statements. Once translated, such statements can be typed into a knowledge base and subsequently used in a program to perform inferencing.

We begin the chapter with an introduction to Propositional Logic, a special case of FOPL. This will be constructive since many of the concepts which apply to this case apply equally well to FOPL. We then proceed in Section 4.3 with a more detailed study of the use of FOPL as a representation scheme. In Section 4.4 we define the syntax and semantics of FOPL and examine equivalent expressions, inference rules, and different methods for mechanized reasoning. The chapter concludes with an example of automated reasoning using a small knowledge base.

SYNTAX AND SEMANTICS FOR PROPOSITIONAL LOGIC

Valid statements or sentences in PL are determined according to the rules of propositional syntax. This syntax governs the combination of basic building blocks such as propositions and logical connectives. Propositions are elementary atomic sentences. (We shall also use the term *formulas* or *well-formed formulas* in place of sentences.) Propositions may be either true or false but may take on no other value. Some examples of simple propositions are

It is raining.
My car is painted silver.
John and Sue have five children.
Snow is white.
People live on the moon.

Compound propositions are formed from atomic formulas using the logical connectives not and or if . . . then, and if and only if. For example, the following are compound formulas.

It is raining and the wind is blowing.
The moon is made of green cheese or it is not.
If you study hard you will be rewarded.
The sum of 10 and 20 is not 50.

We will use capital letters, sometimes followed by digits, to stand for propositions; T and F are special symbols having the values true and false, respectively. The following symbols will also be used for logical connectives

~ for not or negation
& for and or conjunction

V for or or disjunction
→ for if . . . then or implication
↔ for if and only if or double implication

In addition, left and right parentheses, left and right braces, and the period will be used as delimiters for punctuation. So, for example, to represent the compound sentence "It is raining and the wind is blowing" we could write $(R \ \& \ B)$ where R and B stand for the propositions "It is raining" and "the wind is blowing," respectively. If we write $(R \ V \ B)$ we mean "it is raining or the wind is blowing or both" that is, V indicates inclusive disjunction.

Syntax

The syntax of PL is defined recursively as follows.

- T and F are formulas.
  If P and Q are formulas, the following are formulas:

$$\neg P$$

$$(P \ \& \ Q)$$

$$(P \ V \ Q)$$

$$(P \rightarrow Q)$$

$$(P \leftrightarrow Q)$$

All formulas are generated from a finite number of the above operations.

An example of a compound formula is

$$((P \ \& \ \neg Q \ V \ R) \rightarrow (Q \rightarrow S))$$

When there is no chance for ambiguity, we will omit parentheses for brevity: $(\neg(P \ \& \ \neg Q))$ can be written as $\neg(P \ \& \ \neg Q)$. When omitting parentheses, the precedence given to the connectives from highest to lowest is $\neg$, &, V, →, and ↔. So, for example, to add parentheses correctly to the sentence

$$P \ \& \ \neg Q \ V \ R \rightarrow S \leftrightarrow U \ V \ W$$

we write

$$((((P \ \& \ \neg(Q)) \ V \ R) \rightarrow S) \leftrightarrow (U \ V \ W))$$

Semantics

The semantics or meaning of a sentence is just the value true or false; that is, it is an assignment of a truth value to the sentence. The values true and false should not be confused with the symbols T and F which can appear within a sentence. Note however, that we are not concerned here with philosophical issues related to meaning but only in determining the truthfulness or falsehood of formulas when a particular interpretation is given to its propositions. An *interpretation* for a sentence or group of sentences is an assignment of a truth value to each propositional symbol. As an example, consider the statement $(P \ \& \ \neg Q)$. One interpretation (I_1) assigns true to P and false to Q. A different interpretation (I_2) assigns true to P and true to Q. Clearly, there are four distinct interpretations for this sentence.

Once an interpretation has been given to a statement, its truth value can be determined. This is done by repeated application of semantic rules to larger and larger parts of the statement until a single truth value is determined. The semantic rules are summarized in Table 4.1 where t, and t' denote any true statements, f, and f' denote any false statements, and a is any statement.

TABLE 4.1 SEMANTIC RULES FOR STATEMENTS

Rule number	True statements	False statements
1.	T	F
2.	$\neg f$	$\neg t$
3.	$t \ \& \ t'$	$f \ \& \ a$
4.	$t \vee a$	$a \ \& \ f$
5.	$a \vee t$	$f \vee f'$
6.	$a \rightarrow t$	$t \rightarrow f$
7.	$f \rightarrow a$	$t \leftrightarrow f$
8.	$t \leftrightarrow t'$	$f \leftrightarrow t$
9.	$f \leftrightarrow f'$	

We can now find the meaning of any statement given an interpretation I for the statement. For example, let I assign true to P, false to Q and false to R in the statement

$$((P \ \& \ \neg Q) \rightarrow R) \vee Q$$

Application of rule 2 then gives $\neg Q$ as true, rule 3 gives $(P \ \& \ \neg Q)$ as true, rule 6 gives $(P \ \& \ \neg Q) \rightarrow R$ as false, and rule 5 gives the statement value as false.

Properties of Statements

Satisfiable. A statement is satisfiable if there is some interpretation for which it is true.

Contradiction. A sentence is contradictory (unsatisfiable) if there is no interpretation for which it is true.

Valid. A sentence is valid if it is true for every interpretation. Valid sentences are also called tautologies.

Equivalence. Two sentences are equivalent if they have the same truth value under every interpretation

Logical consequences. A sentence is a logical consequence of another if it is satisfied by all interpretations which satisfy the first. More generally, it is a logical consequence of other statements if and only if for any interpretation in which the statements are true, the resulting statement is also true.

A valid statement is satisfiable, and a contradictory statement is invalid, but the converse is not necessarily true. As examples of the above definitions consider the following statements.

P is satisfiable but not valid since an interpretation that assigns false to P assigns false to the sentence P.

$P \vee \neg P$ is valid since every interpretation results in a value of true for $(P \vee \neg P)$.

$P \& \neg P$ is a contradiction since every interpretation results in a value of false for $(P \& \neg P)$.

P and $\neg(\neg P)$ are equivalent since each has the same truth values under every interpretation.

P is a logical consequence of $(P \& Q)$ since any interpretation for which $(P \& Q)$ is true, P is also true.

The notion of logical consequence provides us with a means to perform valid inferencing in PL. The following are two important theorems which give criteria for a statement to be a logical consequence of a set of statements.

Theorem 4.1. The sentence s is a logical consequence of $s_1, \ldots, s_n$ if and only if $s_1 \& s_2 \& \ldots \& s_n \rightarrow s$ is valid.

Theorem 4.2. The sentence s is a logical consequence of $s_1, \ldots, s_n$ if and only if $s_1 \& s_2 \& \ldots \& s_n \& \neg s$ is inconsistent.

The proof of theorem 4.1 can be seen by first noting that if s is a logical consequence of $s_1, \ldots, s_n$, then for any interpretation I in which $s_1 \& s_2 \& \ldots \& s_n$ is true, s is also true by definition. Hence $s_1 \& s_2 \& \ldots \& s_n \rightarrow s$ is true. On the other hand, if $s_1 \& s_2 \& \ldots \& s_n \rightarrow s$ is valid, then for any interpretation I if $s_1 \& s_2 \& \ldots \& s_n$ is true, s is true also.

The proof of theorem 4.2 follows directly from theorem 4.1 since s is a

TABLE 4.2 SOME EQUIVALENCE LAWS

Idempotency	$P \vee P = P$ $P \& P = P$
Associativity	$(P \vee Q) \vee R = P \vee (Q \vee R)$ $(P \& Q) \& R = P \& (Q \& R)$
Commutativity	$P \vee Q = Q \vee P$ $P \& Q = Q \& P$ $P \leftrightarrow Q = Q \leftrightarrow P$
Distributivity	$P \& (Q \vee R) = (P \& Q) \vee (P \& R)$ $P \vee (Q \& R) = (P \vee Q) \& (P \vee R)$
De Morgan's laws	$\neg(P \vee Q) = \neg P \& \neg Q$ $\neg(P \& Q) = \neg P \vee \neg Q$
Conditional elimination	$P \rightarrow Q = \neg P \vee Q$
Bi-conditional elimination	$P \leftrightarrow Q = (P \rightarrow Q) \& (Q \rightarrow P)$

logical consequence of $s_1, \ldots, s_n$ if and only if $s_1 \& s_2 \& \ldots \& s_n \rightarrow s$ is valid, that is, if and only if $\neg(s_1 \& s_2 \& \ldots \& s_n \rightarrow s)$ is inconsistent. But

$$\begin{aligned}\neg(s_1 \& s_2 \& \ldots \& s_n \rightarrow s) &= \neg(\neg(s_1 \& s_2 \& \ldots \& s_n) \vee s) \\ &= \neg\neg(s_1 \& s_2 \& \ldots \& s_n) \& \neg s) \\ &= s_1 \& s_2 \& \ldots \& s_n \& \neg s\end{aligned}$$

When s is a logical consequence of $s_1, \ldots, s_n$, the formula $s_1 \& s_2 \& \ldots \& s_n \rightarrow s$ is called a theorem, with s the conclusion. When s is a logical consequence of the set $S = \{s_1, \ldots, s_n\}$ we will also say S logically implies or logically entails s, written $S \vdash s$.

It is often convenient to make substitutions when considering compound statements. If s_1 is equivalent to s_2, s_1 may be substituted for s_2 without changing the truth value of a statement or set of sentences containing s_2. Table 4.2 lists some of the important laws of PL. Note that the equal sign as used in the table has the same meaning as $\leftrightarrow$; it also denotes equivalence.

One way to determine the equivalence of two sentences is by using truth tables. For example, to show that $P \rightarrow Q$ is equivalent to $\neg P \vee Q$ and that $P \leftrightarrow Q$ is equivalent to the expression $(P \rightarrow Q) \& (Q \rightarrow P)$, a truth table such as Table 4.3, can be constructed to verify or disprove the equivalences.

TABLE 4.3 TRUTH TABLE FOR EQUIVALENT SENTENCES

P	Q	$\neg P$	$(\neg P \vee Q)$	$(P \rightarrow Q)$	$(Q \rightarrow P)$	$(P \rightarrow Q) \& (Q \rightarrow P)$
true	true	false	true	true	true	true
true	false	false	false	false	true	false
false	true	true	true	true	false	false
false	false	true	true	true	true	true

Inference Rules

The inference rules of PL provide the means to perform logical proofs or deductions. The problem is, given a set of sentences $S = \{s_1, \ldots, s_n\}$ (the premises), prove the truth of s (the conclusion); that is, show that $S \vdash s$. The use of truth tables to do this is a form of semantic proof. Other syntactic methods of inference or deduction are also possible. Such methods do not depend on truth assignments but on syntactic relationships only; that is, it is possible to derive new sentences which are logical consequences of $s_1, \ldots, s_n$ using only syntactic operations. We present a few such rules now which will be referred to often throughout the text.

Modus ponens. From P and $P \rightarrow Q$ infer Q. This is sometimes written as

$$\frac{P \qquad P \rightarrow Q}{Q}$$

For example

given: (joe is a father)
and: (joe is a father) → (joe has a child)
conclude: (joe has a child)

Chain rule. From $P \rightarrow Q$, and $Q \rightarrow R$, infer $P \rightarrow R$. Or

$$\frac{P \rightarrow Q \qquad Q \rightarrow R}{P \rightarrow R}$$

For example,

given: (programmer likes LISP) → (programmer hates COBOL)
and: (programmer hates COBOL) → Programmer likes recursion)
conclude: (programmer likes LISP) → (programmer likes recursion)

Substitution. If s is a valid sentence, s' derived from s by consistent substitution of propositions in s, is also valid. For example, the sentence $P \vee \neg P$ is valid; therefore $Q \vee \neg Q$ is also valid by the substitution rule.

Simplification. From $P \,\&\, Q$ infer P.

Conjunction. From P and from Q, infer $P \,\&\, Q$.

Transposition. From $P \rightarrow Q$, infer $\neg Q \rightarrow \neg P$.

We leave it to the reader to justify the last three rules given above.
We conclude this section with the following definitions.

Formal system. A formal system is a set of axioms S and a set of inference rules L from which new statements can be logically derived. We will sometimes denote a formal system as $<S, L>$ or simply a KB (for knowledge base).

Soundness. Let $<S, L>$ be a formal system. We say the inference procedures L are sound if and only if any statements s that can be derived from $<S, L>$ is a logical consequence of $<S, L>$.

Completeness. Let $<S, L>$ be a formal system. Then the inference procedure L is complete if and only if any sentence s logically implied by $<S, L>$ can be derived using that procedure.

As an example of the above definitions, suppose $S = \{P, P \rightarrow Q\}$ and L is the modus ponens rule. Then $<S, L>$ is a formal system, since Q can be derived from the system. Furthermore, this system is both sound and complete for the reasons given above.

We will see later that a formal system like *resolution* permits us to perform computational reasoning. Clearly, soundness and completeness are desirable properties of such systems. Soundness is important to insure that all derived sentences are true when the assumed set of sentences are true. Completeness is important to guarantee that inconsistencies can be found whenever they exist in a set of sentences.

4.3 SYNTAX AND SEMANTICS FOR FOPL

As was noted in the previous chapter, expressiveness is one of the requirements for any serious representation scheme. It should be possible to accurately represent most, if not all concepts which can be verbalized. PL falls short of this requirement in some important respects. It is too "coarse" to easily describe properties of objects, and it lacks the structure to express relations that exist among two or more entities. Furthermore, PL does not permit us to make generalized statements about classes of similar objects. These are serious limitations when reasoning about real world entities. For example, given the following statements, it should be possible to conclude that John must take the Pascal course.

All students in Computer Science must take Pascal.
John is a Computer Science major.

As stated, it is not possible to conclude in PL that John must take Pascal since the second statement does not occur as part of the first one. To draw the desired conclusion with a valid inference rule, it would be necessary to rewrite the sentences.

FOPL was developed by logicians to extend the expressiveness of PL. It is a generalization of PL that permits reasoning about world objects as relational entities as well as classes or subclasses of objects. This generalization comes from the

introduction of predicates in place of propositions, the use of functions and the use of variables together with variable quantifiers. These concepts are formalized below.

The syntax for FOPL, like PL, is determined by the allowable symbols and rules of combination. The semantics of FOPL are determined by interpretations assigned to predicates, rather than propositions. This means that an interpretation must also assign values to other terms including constants, variables and functions, since predicates may have arguments consisting of any of these terms. Therefore, the arguments of a predicate must be assigned before an interpretation can be made.

Syntax of FOPL

The symbols and rules of combination permitted in FOPL are defined as follows.

Connectives. There are five connective symbols: ¬ (not or negation), & (and or conjunction), V (or or inclusive disjunction, that is, A or B or both A and B), → (implication), ↔ (equivalence or if and only if).

Quantifiers. The two quantifier symbols are ∃ (existential quantification) and ∀ (universal quantification), where (∃x) means for some x or there is an x and (∀x) means for all x. When there is no possibility of confusion, we will omit the parentheses for brevity. Furthermore, when more than one variable is being quantified by the same quantifier such as (∀x) (∀y) (∀z) we abbreviate with a single quantifier and drop the parentheses to get ∀xyz.

Constants. Constants are fixed-value terms that belong to a given domain of discourse. They are denoted by numbers, words, and small letters near the beginning of the alphabet such as a, b, c, 5.3, −21, flight-102, and john.

Variables. Variables are terms that can assume different values over a given domain. They are denoted by words and small letters near the end of the alphabet, such as aircraft-type, individuals, x, y, and z.

Functions. Function symbols denote relations defined on a domain D. They map n elements ($n \geq 0$) to a single element of the domain. Symbols f, g, h, and words such as father-of, or age-of, represent functions. An n place (n-ary) function is written as $f(t_1, t_2, \ldots, t_n)$ where the t_i are terms (constants, variables, or functions) defined over some domain. A 0-ary function is a constant.

Predicates. Predicate symbols denote relations or functional mappings from the elements of a domain D to the values true or false. Capital letters and capitalized words such as P, Q, R, EQUAL, and MARRIED are used to represent predicates. Like functions, predicates may have n ($n \geq 0$) terms for arguments written as $P(t_1, t_2, \ldots, t_n)$, where the terms t_i, $i = 1, 2, \ldots, n$ are defined over some domain. A 0-ary predicate is a proposition, that is, a constant predicate.

Constants, variables, and functions are referred to as *terms*, and predicates are referred to as atomic formulas or *atoms* for short. Furthermore, when we want to refer to an atom or its negation, we often use the word *literal*.

In addition to the above symbols, left and right parentheses, square brackets, braces, and the period are used for punctuation in symbolic expressions.

As an example of the above concepts, suppose we wish to represent the following statements in symbolic form.

E1: All employees earning $1400 or more per year pay taxes.
E2: Some employees are sick today.
E3: No employee earns more than the president.

To represent such expressions in FOPL, we must define abbreviations for the predicates and functions. We might, for example, define the following.

$E(x)$ for x is an employee.
$P(x)$ for x is president.
$i(x)$ for the income of x (lower case denotes a function).
$GE(u,v)$ for u is greater than or equal to v.
$S(x)$ for x is sick today.
$T(x)$ for x pays taxes.

Using the above abbreviations, we can represent E1, E2, and E3 as

$$E1': \forall x\ ((E(x)\ \&\ GE(i(x),1400)) \rightarrow T(x))$$
$$E2': \exists y\ (E(y)\ \&\ S(y))$$
$$E3': \forall x y\ ((E(x)\ \&\ P(y)) \rightarrow \neg GE(i(x),i(y)))$$

In the above, we read E1′ as "for all x if x is an employee and the income of x is greater than or equal to $1400 then x pays taxes." More naturally, we read E1′ as E1, that is as "all employees earning $1400 or more per year pay taxes."

E2′ is read as "there is an employee and the employee is sick today" or "some employee is sick today." E3′ reads "for all x and for all y if x is an employee and y is president, the income of x is *not* greater than or equal to the income of y." Again, more naturally, we read E3′ as, "no employee earns more than the president."

The expressions E1′, E2′, and E3′ are known as well-formed formulas or *wffs* (pronounced woofs) for short. Clearly, all of the wffs used above would be more meaningful if full names were used rather than abbreviations.

Wffs are defined recursively as follows:

An atomic formula is a wff.
If P and Q are wffs, then $\neg P$, $P\ \&\ Q$, $P \vee Q$, $P \rightarrow Q$,

$P \leftrightarrow Q$, $\forall x\ P(x)$, and $\exists x\ P(x)$ are wffs.

Wffs are formed only by applying the above rules a finite number of times.

The above rules state that all wffs are formed from atomic formulas and the proper application of quantifiers and logical connectives.

Some examples of valid wffs are

MAN(john)
PILOT(father-of(bill))
∃xyz ((FATHER(x,y) & FATHER(y,z)) → GRANDFATHER(x,z))
∀x NUMBER(x) → (∃y GREATER-THAN(y,x))
∀x ∃y (P(x) & Q(y)) → (R(a) V Q(b))

Some examples of statements that are *not* wffs are:

∀P P(x) → Q(x)
MAN(john)
father-of(Q(x))
MARRIED(MAN,WOMAN)

The first group of examples above are all wffs since they are properly formed expressions composed of atoms, logical connectives, and valid quantifications. Examples in the second group fail for different reasons. In the first expression, universal quantification is applied to the predicate $P(x)$. This is invalid in FOPL.[1] The second expression is invalid since the term John, a constant, is negated. Recall that predicates, and not terms are negated. The third expression is invalid since it is a function with a predicate argument. The last expression fails since it is a predicate with two predicate arguments.

Semantics for FOPL

When considering specific wffs, we always have in mind some domain D. If not stated explicitly, D will be understood from the context. D is the set of all elements or objects from which fixed assignments are made to constants and from which the domain and range of functions are defined. The arguments of predicates must be terms (constants, variables, or functions). Therefore, the domain of each n-place predicate is also defined over D.

For example, our domain might be all entities that make up the Computer Science Department at the University of Texas. In this case, constants would be professors (Bell, Cooke, Gelfond, and so on), staff (Martha, Pat, Linda, and so on), books, labs, offices, and so forth. The functions we may choose might be

[1] Predicates may be quantified in *second* order predicate logic as indicated in the example, but never in first order logic.

advisor-of(x), lab-capacity(y), dept-grade-average(z), and the predicates MARRIED(x), TENURED(y), COLLABORATE(x,y), to name a few.

When an assignment of values is given to each term and to each predicate symbol in a wff, we say an *interpretation* is given to the wff. Since literals always evaluate to either true or false under an interpretation, the value of any given wff can be determined by referring to a truth table such as Table 4.2 which gives truth values for the subexpressions of the wff.

If the truth values for two different wffs are the same under every interpretation, they are said to be *equivalent*. A predicate (or wff) that has no variables is called a *ground atom*.

When determining the truth value of a compound expression, we must be careful in evaluating predicates that have variable arguments, since they evaluate to true only if they are true for the appropriate value(s) of the variables. For example, the predicate $P(x)$ in $\forall x\ P(x)$, is true only if it is true for every value of x in the domain D. Likewise, the $P(x)$ in $\exists x\ P(x)$ is true only if it is true for at least one value of x in the domain. If the above conditions are not satisfied, the predicate evaluates to false.

Suppose, for example, we want to evaluate the truth value of the expression E, where

$$E: \forall x\ ((A(a,x) \vee B(f(x))) \& C(x)) \rightarrow D(x)$$

In this expression, there are four predicates: A, B, C, and D. The predicate A is a two-place predicate, the first argument being the constant a, and the second argument, a variable x. The predicates B, C, and D are all unary predicates where the argument of B is a function $f(x)$, and the argument of C and D is the variable x.

Since the whole expression E is quantified with the universal quantifier $\forall x$, it will evaluate to true only if it evaluates to true for all x in the domain D. Thus, to complete our example, suppose E is interpreted as follows: Define the domain $D = \{1,2\}$ and from D let the interpretation I assign the following values:

a = 2
f(1) = 2, f(2) = 1
A(2,1) = true, A(2,2) = false
B(1) = true, B(2) = false
C(1) = true, C(2) = false
D(1) = false, D(2) = true

Using a table such as Table 4.3 we can evaluate E as follows:

a. If $x = 1$, $A(2,1)$ evaluates to true, $B(2)$ evaluates to false, and $(A(2,1) \vee B(2))$ evaluates to true. $C(1)$ evaluates to true. Therefore, the expression in

the outer parentheses (the antecedent of E) evaluates to true. Hence, since $D(1)$ evaluates to false, the expression E evaluates to false.

b. In a similar way, if $x = 2$, the expression can be shown to evaluate to true. Consequently, since E is not true for all x, the expression E evaluates to false.

4.4 PROPERTIES OF WFFS

As in the case of PL, the evaluation of complex formulas in FOPL can often be facilitated through the substitution of equivalent formulas. Table 4.3 lists a number of equivalent expressions. In the table F, G and H denote wffs not containing variables and $F[x]$ denotes the wff F which contains the variable x. The equivalences can easily be verified with truth tables such as Table 4.3 and simple arguments for the expressions containing quantifiers. Although Tables 4.4 and 4.3 are similar, there are some notable differences, particularly in the wffs containing quantifiers. For example, attention is called to the last four expressions which govern substitutions involving negated quantifiers and the movement of quantifiers across conjunctive and disjunctive connectives.

We summarize here some definitions which are similar to those of the previous section. A wff is said to be *valid* if it is true under every interpretation. A wff that is false under every interpretation is said to be *inconsistent* (or unsatisfiable). A wff that is not valid (one that is false for some interpretation) is *invalid*. Likewise, a wff that is not inconsistent (one that is true for some interpretation) is *satisfiable*. Again, this means that a valid wff is satisfiable and an inconsistent wff is invalid,

TABLE 4.4. EQUIVALENT LOGICAL EXPRESSIONS

$\neg(\neg F) = F$	(double negation)
$F \& G = G \& F$, $F \vee G = G \vee F$	(commutativity)
$(F \& G) \& H = F \& (G \& H)$,	
$(F \vee G) \vee H = F \vee (G \vee H)$	(associativity)
$F \vee (G \& H) = (F \vee G) \& (F \vee H)$,	
$F \& (G \vee H) = (F \& G) \vee (F \& H)$	(distributivity)
$\neg(F \& G) = \neg F \vee \neg G$,	
$\neg(F \vee G) = \neg F \& \neg G$	(De Morgan's Laws)
$F \rightarrow G = \neg F \vee G$	
$F \leftrightarrow G = (\neg F \vee G) \& (\neg G \vee F)$	
$\forall x\, F[x] \vee G = \forall x\, (F[x] \vee G)$,	
$\exists x\, F[x] \vee G = \exists x\, (F[x] \vee G)$	
$\forall x\, F[x] \& G = \forall x\, (F[x] \& G)$,	
$\exists x\, F[x] \& G = \exists x\, (F[x] \& G)$	
$\neg(\forall x)\, F[x] = \exists x\, (\neg F[x])$.	
$\neg(\exists x)\, F[x] = \forall x\, (\neg F[x])$	
$\forall x\, F[x] \& \forall x\, G[x] = \forall x\, (F[x] \& G[x])$	
$\exists x\, F[x] \vee \exists x\, G[x] = \exists x\, (F[x] \vee G[x])$	

but the respective converse statements do not hold. Finally, we say that a wff Q is a *logical consequence* of the wffs $P_1, P_2, \ldots, P_n$ if and only if whenever P_1 & P_2 & . . . & P_n is true under an interpretation, Q is also true.

To illustrate some of these concepts, consider the following examples:

a. P & $\neg P$ is inconsistent and $P \vee \neg P$ is valid since the first is false under every interpretation and the second is true under every interpretation.

b. From the two wffs

$$\text{CLEVER(bill) and}$$
$$\forall x\ \text{CLEVER}(x) \rightarrow \text{SUCCEED}(x)$$

we can show that SUCCEED(bill) is a logical consequence. Thus, assume that both

$$\text{CLEVER(bill) and}$$
$$\forall x\ \text{CLEVER}(x) \rightarrow \text{SUCCEED}(x)$$

are true under an interpretation. Then

$$\text{CLEVER(bill)} \rightarrow \text{SUCCEED(bill)}$$

is certainly true since the wff was assumed to be true for all x, including x = bill. But,

$$\text{CLEVER(bill)} \rightarrow \text{SUCCEED(bill)}$$
$$= \neg\text{CLEVER(bill)} \vee \text{SUCCEED(bill)}$$

are equivalent and, since CLEVER(bill) is true, ¬CLEVER(bill) is false and, therefore, SUCCEED(bill) must be true. Thus, we conclude SUCCEED-(bill) is a logical consequence of

$$\text{CLEVER(bill) and } \forall x\ \text{CLEVER}(x) \rightarrow \text{SUCCEED}(x).$$

Suppose the wff $F[x]$ contains the variable x. We say x is *bound* if it follows or is within the scope of a quantifier naming the variable. If a variable is not bound, it is said to be free. For example, in the expression $\forall x\ (P(x) \rightarrow Q(x,y))$, x is bound, but y is free since every occurrence of x follows the quantifier and y is not within the scope of any quantifier. Clearly, an expression can be evaluated only when all the variables in that expression are bound. Therefore, we shall require that all wffs contain only bound variables. We will also call such expressions a sentence.

We conclude this section with a few more definitions. Given wffs F_1, F_2,

. . . , F_n each possibly consisting of the disjunction of literals only, we say F_1 & F_2 & . . . & F_n is in *conjunctive normal form* (CNF). On the other hand if each F_i, $i = 1, \ldots, n$ consists only of the conjunction of literals, we say $F_1 \vee F_2 \vee \ldots \vee F_n$ is in *disjunctive normal form* (DNF). For example, the wffs $(\neg P \vee Q \vee R) \& (\neg P \vee \neg Q) \& \neg R$ and $(P \& Q \& R) \vee (Q \& R) \vee P$ are in conjunctive and disjunctive normal forms respectively. It can be shown that *any* wff can be transformed into either normal form.

4.5 CONVERSION TO CLAUSAL FORM

As noted earlier, we are interested in mechanical inference by programs using symbolic FOPL expressions. One method we shall examine is called *resolution*. It requires that all statements be converted into a normalized clausal form. We define a *clause* as the disjunction of a number of literals. A *ground clause* is one in which no variables occur in the expression. A *Horn clause* is a clause with at most one positive literal.

To transform a sentence into clausal form requires the following steps:

eliminate all implication and equivalence symbols,

move negation symbols into individual atoms,

rename variables if necessary so that all remaining quantifiers have different variable assignments,

replace existentially quantified variables with special functions and eliminate the corresponding quantifiers,

drop all universal quantifiers and put the remaining expression into CNF (disjunctions are moved down to literals), and

drop all conjunction symbols writing each clause previously connected by the conjunctions on a separate line.

These steps are described in more detail below. But first, we describe the process of eliminating the existential quantifiers through a substitution process. This process requires that all such variables be replaced by something called Skolem functions, arbitrary functions which can always assume a correct value required of an existentially quantified variable.

For simplicity in what follows, assume that all quantifiers have been properly moved to the left side of the expression, and each quantifies a different variable. Skolemization, the replacement of existentially quantified variables with Skolem functions and deletion of the respective quantifiers, is then accomplished as follows:

1. If the first (leftmost) quantifier in an expression is an existential quantifier, replace all occurrences of the variable it quantifies with an arbitrary constant not appearing elsewhere in the expression and delete the quantifier. This same procedure

should be followed for all other existential quantifiers not preceded by a universal quantifier, in each case, using different constant symbols in the substitution.

2. For each existential quantifier that is preceded by one or more universal quantifiers (is within the scope of one or more universal quantifiers), replace all occurrences of the existentially quantified variable by a function symbol not appearing elsewhere in the expression. The arguments assigned to the function should match all the variables appearing in each universal quantifier which precedes the existential quantifier. This existential quantifier should then be deleted. The same process should be repeated for each remaining existential quantifier using a different function symbol and choosing function arguments that correspond to all universally quantified variables that precede the existentially quantified variable being replaced.

An example will help to clarify this process. Given the expression

$$\exists u\ \forall v\ \forall x\ \exists y\ P(f(u), v, x, y) \rightarrow Q(u,v,y)$$

the Skolem form is determined as

$$\forall v\ \forall x\ P(f(a),v,x,g(v,x)) \rightarrow Q(a,v,g(v,x)).$$

In making the substitutions, it should be noted that the variable u appearing after the first existential quantifier has been replaced in the second expression by the arbitrary constant a. This constant did not appear elsewhere in the first expression. The variable y has been replaced by the function symbol g having the variables v and x as arguments, since both of these variables are universally quantified to the left of the existential quantifier for y. Replacement of y by an arbitrary function with arguments v and x is justified on the basis that y, following v and x, may be functionally dependent on them and, if so, the arbitrary function g can account for this dependency. The complete procedure can now be given to convert any FOPL sentence into clausal form.

Clausal Conversion Procedure

Step 1. Eliminate all implication and equivalency connectives (use $\neg P \vee Q$ in place of $P \rightarrow Q$ and $(\neg P \vee Q) \,\&\, (\neg Q \vee P)$ in place of $P \leftrightarrow Q$.

Step 2. Move all negations in to immediately precede an atom (use P in place of $\neg(\neg P)$, and DeMorgan's laws, $\exists x\ \neg F[x]$ in place of $\neg(\forall x)\ F[x]$ and $\forall x\ \neg F[x]$ in place of $\neg(\exists x)\ F[x]$).

Step 3. Rename variables, if necessary, so that all quantifiers have different variable assignments; that is, rename variables so that variables bound by one quantifier are not the same as variables bound by a different quantifier. For example, in the expression $\forall x\ (P(x) \rightarrow (\exists x\ (Q(x))))$ rename the second "dummy" variable x which is bound by the existential quantifier to be a different variable, say y, to give $\forall x\ (P(x) \rightarrow (\exists y\ Q(y)))$.

Step 4. Skolemize by replacing all existentially quantified variables with Skolem functions as described above, and deleting the corresponding existential quantifiers.

Step 5. Move all universal quantifiers to the left of the expression and put the expression on the right into CNF.

Step 6. Eliminate all universal quantifiers and conjunctions since they are retained implicitly. The resulting expressions (the expressions previously connected by the conjunctions) are clauses and the set of such expressions is said to be in clausal form.

As an example of this process, let us convert the expression

$$\exists x\, \forall y\, (\forall z\, P(f(x),y,z) \rightarrow (\exists u\, Q(x,u)\ \&\ \exists v\, R(y,v)))$$

into clausal form. We have after application of step 1

$$\exists x\, \forall y\, \neg(\forall z)\, P(f(x),y,z) \vee (\exists u\, Q(x,u)\ \&\ (\exists v)\, R(y,v)))).$$

After application of step 2 we obtain

$$\exists x\, \forall y\, (\exists z\, \neg P(f(x),y,z) \vee (\exists u\, Q(x,u)\ \&\ (\exists v)\, R(y,v)))).$$

After application of step 4 (step 3 is not required)

$$\forall y\, \neg P(f(a),y,g(y)) \vee (Q(a,h(y))\ \&\ R(y,l(y)))).$$

After application of step 5 the result is

$$\forall y\, ((\neg P(f(a),y,g(y)) \vee Q(a,h(y))\ \&\ \neg P(f(a),y,g(y)) \vee R(y,l(y)))).$$

Finally, after application of step 6 we obtain the clausal form

$$\neg P(f(a),y,g(y)) \vee Q(a,h(y))$$
$$\neg P(f(a),y,g(y) \vee R(y,l(y))$$

The last two clauses of our final form are understood to be universally quantified in the variable y and to have the conjunction symbol connecting them.

It should be noted that the set of clauses produced by the above process are not *equivalent* to the original expression, but *satisfiability* is retained. That is, the set of clauses are satisfiable if and only if the original sentence is satisfiable.

Having now labored through the tedious steps above, we point out that it is often possible to write down statements directly in clausal form without working through the above process step-by-step. We illustrate how this may be done in Section 4.7 when we create a sample knowledge base.

4.6 INFERENCE RULES

Like PL, a key inference rule in FOPL is modus ponens. From the assertion "Leo is a lion" and the implication "all lions are ferocious" we can conclude that Leo is ferocious. Written in symbolic form we have

assertion: LION(leo)
implication: $\forall x$ LION(x) $\rightarrow$ FEROCIOUS(x)
conclusion: FEROCIOUS(leo)

In general, if a has property P and all objects that have property P also have property Q, we conclude that a has property Q.

$$\frac{P(a) \qquad \forall x\, P(x) \rightarrow Q(x)}{Q(a)}$$

Note that in concluding $Q(a)$, a substitution of a for x was necessary. This was possible, of course, since the implication $P(x) \rightarrow Q(x)$ is assumed true for all x, and in particular for $x = a$. Substitutions are an essential part of the inference process. When properly applied, they permit simplifications or the reduction of expressions through the cancellation of complementary literals. We say that two literals are *complementary* if they are identical but of opposite sign; that is, P and $\neg P$ are complementary.

A *substitution* is defined as a set of pairs t_i and v_i where v_i are distinct variables and t_i are terms not containing the v_i. The t_i replace or are substituted for the corresponding v_i in any expression for which the substitution is applied. A set of substitutions $\{t_1/v_1, t_2/v_2, \ldots, t_n/v_n\}$ where $n \geq 1$ applied to an expression will be denoted by Greek letters α, β, and δ. For example, if $\beta = \{a/x, g(b)/y\}$, then applying β to the clause $C = P(x,y) \vee Q(x,f(y))$ we obtain $C' = C\beta = P(a,g(b)) \vee Q(a,f(g(b)))$.

Unification

Any substitution that makes two or more expressions equal is called a *unifier* for the expressions. Applying a substitution to an expression E produces an *instance* E' of E where $E' = E\beta$. Given two expressions that are unifiable, such as expressions C_1 and C_2 with a unifer β with $C_1\beta = C_2$, we say that β is a *most general unifer* (mgu) if any other unifer α is an instance of β. For example two unifiers for the literals $P(u,b,v)$ and $P(a,x,y)$ are $\alpha = \{a/u,b/x,v/y\}$ and $\beta = \{a/u,b/x,c/v,c/y\}$. The former is an mgu whereas the latter is not since it is an *instance* of the former.

Unification can sometimes be applied to literals within the same single clause. When an mgu exists such that two or more literals within a clause are unified, the clause remaining after deletion of all but one of the unified literals is called a

factor of the original clause. Thus, given the clause $C = P(x) \vee Q(x,y) \vee P(f(z))$ the factor $C' = C\beta = P(f(z)) \vee Q(f(z),y)$ is obtained where $\beta = \{f(z)/x\}$.

Let **S** be a set of expressions. We define the *disagreement set* of **S** as the set obtained by comparing each symbol of all expressions in **S** from left to right and extracting from **S** the subexpressions whose first symbols do not agree. For example, let $\mathbf{S} = \{P(f(x),g(y),a), P(f(x),z,a), P(f(x),b,h(u))\}$. For the set **S**, the disagreement set is $\{g(y),a,b,z,h(u)\}$. We can now state a unification algorithm which returns the mgu for a given set of expressions **S**.

Unification algorithm:

1. Set $k = 0$ and $\sigma_k = e$ (the empty set).
2. If the set $\mathbf{S}\sigma_k$ is a singleton, then stop; σ_k is an mgu of **S**. Otherwise, find the disagreement set D_k of $\mathbf{S}\sigma_k$.
3. If there is a variable v and term t in D_k such that v does not occur in t, put $\sigma_{k+1} = \sigma_k\{t/v\}$, set $k = k + 1$, and return to step 2. Otherwise, stop. **S** is not unifiable.

4.7 THE RESOLUTION PRINCIPLE

We are now ready to consider the resolution principle, a syntactic inference procedure which, when applied to a set of clauses, determines if the set is unsatisfiable. This procedure is similar to the process of obtaining a proof by contradiction. For example, suppose we have the set of clauses (axioms) $C_1, C_2, \ldots, C_n$ and we wish to deduce or prove the clause D, that is, to show that D is a logical consequence of $C_1 \,\&\, C_2 \,\&\, \ldots \,\&\, C_n$. First, we negate D and add $\neg D$ to the set of clauses $C_1, C_2, \ldots, C_n$. Then, using resolution together with factoring, we can show that the set is unsatisfiable by deducing a contradiction. Such a proof is called a proof by refutation which, if successful, yields the empty clause denoted by [ ].[2] Resolution with factoring is *complete* in the sense that it will always generate the empty clause from a set of unsatisfiable clauses.

Resolution is very simple. Given two clauses C_1 and C_2 with no variables in common, if there is a literal l_1 in C_1 which is a complement of a literal l_2 in C_2, both l_1 and l_2 are deleted and a disjuncted C is formed from the remaining reduced clauses. The new clause C is called the *resolvent* of C_1 and C_2. Resolution is the process of generating these resolvents from a set of clauses. For example, to resolve the two clauses

$$(\neg P \vee Q) \text{ and } (\neg Q \vee R)$$

[2] The empty clause [ ] is always false since no interpretation can satisfy it. It is derived from combining contradictory clauses such as P and $\neg P$.

we write

$$\frac{\neg P \vee Q,\ \neg Q \vee R}{\neg P \vee R}$$

Several types of resolution are possible depending on the number and types of parents. We define a few of these types below.

Binary resolution. Two clauses having complementary literals are combined as disjuncts to produce a single clause after deleting the complementary literals. For example, the binary resolvent of

$$\neg P(x,a) \vee Q(x) \quad \text{and} \quad \neg Q(b) \vee R(x)$$

is just

$$\neg P(b,a) \vee R(b).$$

The substitution $\{b/x\}$ was made in the two parent clauses to produce the complementary literals $Q(b)$ and $\neg Q(b)$ which were then deleted from the disjunction of the two parent clauses.

Unit resulting (UR) resolution. A number of clauses are resolved simultaneously to produce a unit clause. All except one of the clauses are unit clauses, and that one clause has exactly one more literal than the total number of unit clauses. For example, resolving the set

$$\{\neg \text{MARRIED}(x,y) \vee \neg \text{MOTHER}(x,z) \vee \text{FATHER}(y,z),\ \text{MARRIED}(\text{sue},\text{joe}),\ \neg \text{FATHER}(\text{joe},\text{bill})\}$$

where the substitution $\beta = \{\text{sue}/x,\ \text{joe}/y,\ \text{bill}/z\}$ is used, results in the unit clause $\neg \text{MOTHER}(\text{sue},\text{bill})$.

Linear resolution. When each resolved clause C_i is a parent to the clause C_{i+1} $(i = 1, 2, \ldots, n-1)$ the process is called linear resolution. For example, given a set S of clauses with $C_0 \subseteq S$, C_n is derived by a sequence of resolutions, C_0 with some clause B_0 to get C_1, then C_1 with some clause B_1 to get C_2, and so on until C_n has been derived.

Linear input resolution. If one of the parents in linear resolution is always from the original set of clauses (the B_i), we have linear input resolution. For example, given the set of clauses $S = \{P \vee Q,\ \neg P \vee Q,\ P \vee \neg Q,\ \neg P \vee \neg Q\}$ let $C_0 = (P \vee Q)$. Choosing $B_0 = \neg P \vee Q$ from the set S and resolving this with C_0 we obtain the resolvent $Q = C_1$. B_1 must now be chosen from S and the resolvent of C_1 and B_1 becomes C_2 and so on.

Unification and resolution give us one approach to the problem of mechanical inference or automated reasoning, but without some further refinements, resolution

can be intolerably inefficient. Randomly resolving clauses in a large set can result in inefficient or even impossible proofs. Typically, the curse of combinatorial explosion occurs. So methods which constrain the search in some way must be used.

When attempting a proof by resolution, one ideally would like a minimally unsatisfiable set of clauses which includes the conjectured clause. A *minimally unsatisfiable set* is one which is satisfiable when any member of the set is omitted. The reason for this choice is that irrelevant clauses which are not needed in the proof but which participate are unnecessary resolutions. They contribute nothing toward the proof. Indeed, they can sidetrack the search direction resulting in a dead end and loss of resources. Of course, the set must be unsatisfiable otherwise a proof is impossible.

A minimally unsatisfiable set is ideal in the sense that all clauses are essential and no others are needed. Thus, if we wish to prove B, we would like to do so with a set of clauses $S = \{A_1, A_2, \ldots, A_k\}$ which become minimally unsatisfiable with the addition of $\neg B$.

Choosing the order in which clauses are resolved is known as a search strategy. While there are many such strategies now available, we define only one of the more important ones, the set-of-support strategy. This strategy separates a set which is unsatisfiable into subsets, one of which is satisfiable.

Set-of-support strategy. Let S be an unsatisfiable set of clauses and T be a subset of S. Then T is a set-of-support for S if $S - T$ is satisfiable. A set-of-support resolution is a resolution of two clauses not both from $S - T$. This essentially means that given an unsatisfiable set $\{A_1, \ldots, A_k\}$, resolution should not be performed directly among the A_i as noted above.

Example of Resolution

The example we present here is one to which all AI students should be exposed at some point in their studies. It is the famous "monkey and bananas problem," another one of those complex real life problems solvable with AI techniques. We envision a room containing a monkey, a chair, and some bananas that have been hung from the center of the ceiling, out of reach from the monkey. If the monkey is clever enough, he can reach the bananas by placing the chair directly below them and climbing on top of the chair. The problem is to use FOPL to represent this monkey-banana world and, using resolution, prove the monkey can reach the bananas.

In creating a knowledge base, it is essential first to identify all relevant objects which will play some role in the anticipated inferences. Where possible, irrelevant objects should be omitted, but never at the risk of incompleteness. For example, in the current problem, the monkey, bananas, and chair are essential. Also needed is some reference object such as the floor or ceiling to establish the height relationship between monkey and bananas. Other objects such as windows, walls or doors are not relevant.

The next step is to establish important properties of objects, relations between

them, and any assertions likely to be needed. These include such facts as the chair is tall enough to raise the monkey within reach of the bananas, the monkey is dexterous, the chair can be moved under the bananas, and so on. Again, all important properties, relations, and assertions should be included and irrelevant ones omitted. Otherwise, unnecessary inference steps may be taken.

The important factors for our problem are described below, and all items needed for the actual knowledge base are listed as axioms. These are the essential facts and rules. Although not explicitly indicated, all variables are universally quantified.

Relevant factors for the problem

CONSTANTS

{floor, chair, bananas, monkey}

VARIABLES

{x, y, z}

PREDICATES

```
{can_reach(x,y)      ; x can reach y
dexterous (x)        ; x is a dexterous animal
close(x,y)           ; x is close to y
get_on(x,y)          ; x can get on y
under(x,y)           ; x is under y
tall(x)              ; x is tall
in_room(x)           ; x is in the room
can_move(x,y,z)      ; x can move y near z
can_climb(x,y)}      ; x can climb onto y
```

AXIOMS

```
{in_room(bananas)
in_room(chair)
in_room(monkey)
dexterous(monkey)
tall(chair)
 close(bananas,floor)
can_move(monkey,chair,bananas)
can_climb(monkey,chair)
(dexterous(x) & close(x,y) → can-reach(x,y)
((get_on(x,y) & under(y,bananas) & tall(y) →
   close(x,bananas))
((in_room(x) & in_room(y) & in_room(z) & can_move(x,y,z))
    → close(z,floor) V under(y,z))
(can_climb(x,y) → get_on(x,y))}
```

Using the above axioms, a knowledge base can be written down directly in the required clausal form. All that is needed to make the necessary substitutions are the equivalences

$$P \rightarrow Q = \neg P \vee Q$$

and De Morgan's laws. To relate the clauses to a LISP program, one may prefer to think of each clause as being a list of items. For example, number 9, below, would be written as

```
(or (~can_climb(?x ?y) get_on(?x ?y))
```

where ?*x* and ?*y* denote variables.

Note that clause 13 is not one of the original axioms. It has been added for the proof as required in refutation resolution proofs.

Clausal form of knowledge base

1. in_room(monkey)
2. in_room(bananas)
3. in_room(chair)
4. tall(chair)
5. dexterous(monkey)
6. can_move(monkey, chair, bananas)
7. can_climb(monkey,chair)
8. ~close(bananas,floor)
9. ~can_climb(x,y) V get_on(x,y)
10. ~dexterous(x) V ~close(x,y) V can_reach(x,y)
11. ~get_on(x,y) V ~under(y,bananas) V ~tall(y) V close(x,bananas)
12. ~in_room(x) V ~in_room(y) V ~in_room(z) V ~can_move(x,y,z) V close(y,floor) V under(y,z)
13. ~can_reach(monkey,bananas)

Resolution proof. A proof that the monkey can reach the bananas is summarized below. As can be seen, this is a refutation proof where the statement to be proved (can_reach(monkey,bananas)) has been negated and added to the knowledge base (number 13). The proof then follows when a contradiction is found (see number 23, below).

14. ~can_move(monkey,chair,bananas) V close(bananas,floor) V under (chair,bananas) ; 14 is a resolvent of 1,2,3 and 12 with substitution (monkey/*x*, chair *y*, bananas/*z*)

15.	close(bananas,floor) V under (chair,bananas)	; this is a resolvent of 6 and 14
16.	under(chair,bananas)	; this is a resolvent of 8 and 15
17.	~get_on(x,chair) V ~tall(chair) V close(x,bananas)	; this is a resolvent of 11 and 16 with substitution {chair/y}
18.	~get_on(x,chair) V close(x,bananas)	; a resolvent of 4 and 17
19.	get_on(monkey,chair)	; a resolvent of 7 and 9
20.	close(monkey,bananas)	; a resolvent of 18 and 19 with substitution {monkey/x}
21.	~close(monkey,y) V can_reach (monkey,y)	; a resolvent of 10 and 5 with substitution {monkey/x}
22.	reach(monkey,bananas)	; a resolvent of 20 and 21 with substitution {bananas/y}
23.	[]	; a resolvent of 13 and 22

In performing the above proof, no particular strategy was followed. Clearly, however, good choices were made in selecting parent clauses for resolution. Otherwise, many unnecessary steps may have been taken before completing the proof. Different forms of resolution were completed in steps 14 through 23. One of the exercises requires that the types of resolutions used be identified.

The Monkey-Banana Problem in PROLOG

Prolog was introduced in the previous chapter. It is a logic programming language that is based on the resolution principle. The refutation proof strategy used in PROLOG is resolution by selective linear with definite clauses or SLD resolution. This is just a form of linear input resolution using definite Horn clauses (clauses with exactly one positive literal). In finding a resolution proof, PROLOG searches a data base of clauses in an exhaustive manner (guided by the SLD strategy) until a chain of unifications have been found to produce a proof.

Next, we present a PROLOG program for the monkey-banana problem to illustrate the ease with which many logic programs may be formulated.

```
% Constants:
% {floor, chair, bananas, monkey}

% Variables:
% {X, Y, Z}

% Predicates:
% {can-reach(X,Y)        ; X can reach Y
%  dexterous(X)          ; X is a dexterous animal
%  close(X,Y)            ; X is close to Y
```

```
% get-on(X,Y)          ; X can get on Y
% under(X,Y)           ; X is under Y
% tall(X)              ; X is tall
% in-room(X)           ; X is in the room
% can-move(X,Y,Z)      ; X can move Y near Z
% can-climb(X,Y)}      ; X can climb onto Y

% Axioms :

in-room(bananas).

in-room(chair).

in-room(monkey).

dexterous(monkey).

tall(chair).

can-move(monkey, chair, bananas).

can-climb(monkey, chair).

can-reach(X,Y) :-

        dexterous(X), close(X,Y).

close(X,Z) :-

        get-on(X,Y),

        under(Y,Z),

        tall(Y).

get-on(X,Y) :-

        can-climb(X,Y).

Under(Y,Z) :-

        in-room(X),

        in-room(Y),

        in-room(Z),

        can-move(X,Y,Z).
```

This completes the data base of facts and rules required. Now we can pose various queries to our theorem proving system.

```
| ?- can-reach(X,Y).

X = monkey,

Y = bananas

| ?- can-reach(X,bananas).

X = monkey

| ?- can-reach(monkey,Y).

Y = bananas

| ?- can-reach(monkey,bananas).

yes

| ?- can-reach(lion,bananas).

no

| ?- can-reach(monkey,apple).

no
```

4.8 NONDEDUCTIVE INFERENCE METHODS

In this section we consider three nondeductive forms of inferencing. These are not valid forms of inferencing, but they are nevertheless very important. We use all three methods often in every day activities where we draw conclusions and make decisions. The three methods we consider here are abduction, induction, and analogical inference.

Abductive Inference

Abductive inference is based on the use of known causal knowledge to explain or justify a (possibly invalid) conclusion. Given the truth of proposition Q and the implication $P \rightarrow Q$, conclude P. For example, people who have had too much to drink tend to stagger when they walk. Therefore, it is not unreasonable to conclude that a person who is staggering is drunk even though this may be an incorrect

conclusion. People may stagger when they walk for other reasons, including dizziness from twirling in circles or from some physical problem.

We may represent abductive inference with the following, where the c over the implication arrow is meant to imply a possible causal relationship.

$$\begin{array}{ll} \text{assertion} & Q \\ \text{implication} & P \overset{c}{\rightarrow} Q \\ \text{conclusion} & \overline{P} \end{array}$$

Abductive inference is useful when known causal relations are likely and deductive inferencing is not possible for lack of facts.

Inductive Inference

Inductive inferencing is based on the assumption that a recurring pattern, observed for some event or entity, implies that the pattern is true for all entities in the class. Given instances $P(a_1)$, $P(a_2)$, . . . , $P(a_k)$, conclude that $\forall x\, P(x)$. More generally, given $P(a_1) \rightarrow Q(b_1)$, $P(a_2) \rightarrow Q(b_2)$, . . . , $P(a_k) \rightarrow Q(b_k)$, conclude $\forall x,y\, P(x) \rightarrow Q(y)$.

We often make this form of generalization after observing only a few instances of a situation. It is known as the *inductive leap*. For example, after seeing a few white swans, we incorrectly infer that all swans are white (a type of Australian swan is black), or we conclude that all Irishmen are stubborn after discussions with only a few.

We can represent inductive inference using the following description:

$$\frac{P(a_1), \ldots, P(a_k)}{\forall x\, P(x)}$$

Inductive inference, of course, is not a valid form of inference, since it is not usually the case that all objects of a class can be verified as having a particular property. Even so, this is an important and commonly used form of inference.

Analogical Inference

Analogical inference is a form of experiential inference. Situations or entities which are alike in some respects tend to be similar in other respects. Thus, when we find that situation (object) A is related in certain ways to B, and A' is similar in some context to A, we conclude that B' has a similar relation to A' in this context. For example, to solve a problem with three equations in three unknowns, we try to extend the methods we know in solving two equations in two unknowns.

Analogical inference appears to be based on the use of a combination of three other methods of inference, abductive, deductive and inductive. We depict this form of inference with the following description, where the r above the implication symbol means is related to.

$$\frac{P' \rightarrow Q}{P'' \rightarrow Q'}$$

Analogical inference, like abductive and inductive is a useful but invalid form of commonsense inference.

4.9 REPRESENTATIONS USING RULES

Rules can be considered a subset of predicate logic. They have become a popular representation scheme for expert systems (also called rule-based systems). They were first used in the General Problem Solver system in the early 1970s (Newell and Simon, 1972).

Rules have two component parts: a left-hand side (LHS) referred to as the antecedent, premise, condition, or situation, and a right-hand side (RHS) known as the consequent, conclusion, action, or response. The LHS is also known as the if part and the RHS as the then part of the rule. Some rules also include an else part. Examples of rules which might be used in expert systems are given below.

IF: The temperature is greater than 95 degrees C.
THEN: Open the relief valve.

IF: The patient has a high temperature,
and the stain is gram-positive,
and the patient has a sore throat,
THEN: The organism is streptococcus.

IF: The lights do not come on,
and the engine does not turn over,
THEN: The battery is dead or the cable is loose.

IF: $A \ \& \ B \ \& \ C$
THEN: D
$A \ \& \ B \ \& \ (C \vee D) \rightarrow D$

The simplest form of a rule-based production system consists of three parts, a knowledge base (KB) consisting of a set of rules (as few as 50 or as many as several thousand rules may be required in an expert system), a working memory, and a rule interpreter or inference engine. The interpreter inspects the LHS of each rule in the KB until one is found which matches the contents of working memory. This causes the rule to be activated or to "fire" in which case the contents of working memory are replaced by the RHS of the rule. The process continues by scanning the next rules in sequence or restarting at the beginning of the knowledge base.

INTERNAL FORM

```
RULE047
    Premise:  (($and (same cntxt site blood)
                     (notdefinite contxt ident)
                     (same cntxt morph rod)
                     (same cntxt burn t))
    Action:   (conclude cntxt ident pseudonomas 0.4))
```

ENGLISH TRANSLATION

IF: 1) The site of the culture is blood, and
2) The identity of the organism is not known with certainty, and
3) The stain of the organism is gramneg, and
4) The morphology of the organism is rod, and
5) The patient has been seriously burned

THEN: There is weakly suggestive evidence (0.4) that the identity of the organism is pseudonomas.

Figure 4.1 A rule from the MYCIN system.

Each rule represents a chunk of knowledge as a conditional statement, and each invocation of the rules as a sequence of actions. This is essentially an inference process using the chain rule as described in Section 4.2. Although there is no established syntax for rule-based systems, the most commonly used form permits a LHS consisting of a conjunction of several conditions and a single RHS action term.

An example of a rule used in the MYCIN[3] expert system (Shortliffe, 1976) is illustrated in Figure 4.1.

In RULE047, the quantity 0.4 is known as a confidence factor (CF). Confidence factors range from -1.0 (complete disbelief) to 1.0 (certain belief). They provide a measure of the experts' confidence that a rule is applicable or holds when the conditions in the LHS have all been satisfied.

We will see further examples of rules and confidence factors in later chapters.

4.10 SUMMARY

We have considered propositional and first order predicate logics in this chapter as knowledge representation schemes. We learned that while PL has a sound theoretical foundation, it is not expressive enough for many practical problems. FOPL, on the

[3] MYCIN was one of the earliest expert systems. It was developed at Stanford University in the mid-1970s to demonstrate that a system could successfully perform diagnoses of patients having infectious blood diseases.

other hand, provides a theoretically sound basis and permits a great latitude of expressiveness. In FOPL one can easily code object descriptions and relations among objects as well as general assertions about classes of similar objects. The increased generality comes from the joining of predicates, functions, variables, and quantifiers. Perhaps the most difficult aspect in using FOPL is choosing appropriate functions and predicates for a given class of problems.

Both the syntax and semantics of PL and FOPL were defined and examples given. Equivalent expressions were presented and the use of truth tables was illustrated to determine the meaning of complex formulas. Rules of inference were also presented, providing the means to derive conclusions from a basic set of facts or axioms. Three important syntactic inference methods were defined: modus ponens, chain rule and resolution. These rules may be summarized as follows.

MODUS PONENS	CHAIN RULE	RESOLUTION
$\dfrac{P \quad P \rightarrow Q}{Q}$	$\dfrac{P \rightarrow Q \quad Q \rightarrow R}{P \rightarrow R}$	$\dfrac{P \vee \neg Q, \; Q \vee R}{P \vee R}$

A detailed procedure was given to convert any complex set of formulas to clausal normal forms. To perform automated inference or theorem proving, the resolution method requires that the set of axioms and the conjecture to be proved be in clausal form. Resolution is important because it provides the means to mechanically derive conclusions that are valid. Programs using resolution methods have been developed for numerous systems with varying degrees of success, and the language PROLOG is based on the use of such resolution proofs. FOPL has without question become one of the leading methods for knowledge representation.

In addition to valid forms of inference based on deductive methods, three invalid, but useful types of inferencing were also presented, abductive, inductive, and analogical.

Finally, rules, a subset of FOPL, were described as a popular representation scheme. As will be seen in Chapter 15, many expert systems use rules for their knowledge bases. Rules provide a convenient means of incrementally building a knowledge base in an easily understood language.

EXERCISES

4.1 Construct a truth table for the expression $(A \; \& \; (A \vee B))$. What single term is this expression equivalent to?

4.2 Prove the following rules from Section 4.2.

(a) Simplification: From $P \; \& \; Q$, infer P

(b) Conjunction: From P and Q, infer $P \; \& \; Q$

(c) Transposition: From $P \rightarrow Q$, infer $\neg Q \rightarrow \neg P$

4.3 Given the following PL expressions, place parentheses in the appropriate places to form fully abbreviated wffs.

(a) $\neg P \vee Q \,\&\, R \rightarrow S \rightarrow U \,\&\, Q$

(b) $\neg P \,\&\, Q \vee P \leftrightarrow U \rightarrow \neg R$

(c) $Q \vee P \vee \neg R \,\&\, S \rightarrow \neg U \,\&\, P \leftrightarrow R$

4.4 Translate the following axioms into predicate calculus wffs. For example, A_1 below could be given as

$$\forall x,y,z \quad \text{CONNECTED}(x,y,z) \,\&\, \text{Bikesok}(z) \rightarrow \text{GETTO}(x,y)$$

A1. If town x is connected to town y by highway z, and bicycles are allowed on z, you can get to y from x by bike.

A2. If town x is connected to y by z, y is connected to x by z.

A3. If you can get to y from x, and you can get to z from y, you can get to z from x.

A4. Town-A is connected to Town-B by Road-1.

A5. Town-B is connected to Town-C by Road-2.

4.5 Convert axioms A1-A5 from Problem 4.4 to clausal form and write axioms A6-A13, below, directly into clausal form.

A6. Town-A is connected to Town-C by Road-3.

A7. Town-D is connected to Town-E by Road-4.

A8. Town-D is connected to Town-B by Road-5.

A9-A11. Bikes are allowed on Road-3, Road-4, and Road-5.

A12. Bikes are always allowed on either Road-2 or Road-1 (each day it may be different but one road is always possible).

A13. Town-A and Town-E are not connected by Road-3.

4.6 Use the clauses from Problem 4.5 to answer the following questions:

(a) What can be deduced from clauses 2 and 13?

(b) Can GETTO(Town-D, Town-B) be deduced from the above clauses?

4.7 Find the meaning of the statement

$$(\neg P \vee Q) \,\&\, R \rightarrow S \vee (\neg \,\&\, Q)$$

for each of the interpretations given below.

(a) I_1: P is true, Q is true, R is false, S is true.

(b) I_2: P is true, Q is false, R is true, S is true.

4.8 Transform each of the following sentences into disjunctive normal form.

(a) $\neg(P \,\&\, Q) \,\&\, (P \vee Q)$

(b) $\neg(P \vee \neg Q) \,\&\, (R \rightarrow S)$

(c) $P \rightarrow ((Q \,\&\, R) \leftrightarrow S)$

4.9 Transform each of the following sentences into conjunctive normal form.

(a) $(P \rightarrow Q) \rightarrow R$

(b) $P \vee (\neg P \,\&\, Q \,\&\, R)$

(c) $(\neg P \,\&\, Q) \vee (P \,\&\, \neg Q) \,\&\, S$

4.10 Determine whether each of the following sentences is

(a) satisfiable

(b) contradictory

(c) valid

S_1: $(P \& Q) \vee \neg(P \& Q)$ S_2: $(P \vee Q) \rightarrow (P \& Q)$
S_3: $(P \& Q) \rightarrow R \vee \neg Q$ S_4: $(P \vee Q) \& (P \vee \neg Q) \vee P$
S_5: $P \rightarrow Q \rightarrow \neg P$ S_6: $P \vee Q \& \neg P \vee \neg Q \& P$

4.11 Given the following wffs $P \rightarrow Q$, $\neg Q$, and $\neg P$, show that $\neg P$ is a logical consequene of the two preceding wffs:
(a) Using a truth table
(b) Using Theorem 4.1.

4.12 Given formulas S_1 and S_2 below, show that $Q(a)$ is a logical consequence of the two.

$$S_1: (\forall x)(P(x) \rightarrow Q(x)) \qquad S_2: P(a)$$

4.13 Transform the following formula to prenex normal form

$$\forall xy\ (\exists z\ P(x,z)\ \&\ P(y,z)) \rightarrow \exists u\ Q(x,y,u)$$

4.14 Prove that the formula $\exists x\ P(x) \rightarrow \forall x\ P(x)$ is always true if the domain D contains only one element.

4.15 Find the standard normal form for the following statement.

$$\forall x\ (\neg P(x,0) \rightarrow (\exists y\ (P(y,g(x))\ \&\ \forall z\ (P(z,g(x)) \rightarrow P(y,z))$$

4.16 Referring to Problems 4.4 and 4.5, use resolution to show (by refutation) that you can get from Town-E to Town-C.

4.17 Write a LISP resolution program to answer different queries about a knowledge base where the knowledge base and queries are given in clausal form. For example,

```
(setq KB (A1 to A13))
(query (GETTO Town-E Town-C))
```

4.18 Given the following information for a database:
A1. If x is on top of y, y supports x.
A2. If x is above y and they are touching each other, x is on top of y.
A3. A cup is above a book.
A4. A cup is touching a book.
(a) Translate statements A1 through A4 into clausal form.
(b) Show that the predicate supports (book, cup) is true using resolution.

4.19 Write a PROLOG program using A1 to A4 of Problem 4.18, and show that SUPPORTS (book cup) is true.

5

Dealing with Inconsistencies and Uncertainties

The previous chapter considered methods of reasoning under conditions of certain, complete, unchanging, and consistent facts. It was implicitly assumed that a sufficient amount of reliable knowledge (facts, rules, and the like) was available with which to deduce confident conclusions. While this form of reasoning is important, it suffers from several limitations.

1. It is not possible to describe many envisioned or real-world concepts; that is, it is limited in expressive power.
2. There is no way to express uncertain, imprecise, hypothetical or vague knowledge, only the truth or falsity of such statements.
3. Available inference methods are known to be inefficient.
4. There is no way to produce new knowledge about the world. It is only possible to add what is derivable from the axioms and theorems in the knowledge base.

In other words, strict classical logic formalisms do not provide realistic representations of the world in which we live. On the contrary, intelligent beings are continuously required to make decisions under a veil of uncertainty.

Uncertainty can arise from a variety of sources. For one thing, the information

we have available may be incomplete or highly volatile. Important facts and details which have a bearing on the problems at hand may be missing or may change rapidly. In addition, many of the "facts" available may be imprecise, vague, or fuzzy. Indeed, some of the available information may be contradictory or even unbelievable. However, despite these shortcomings, we humans miraculously deal with uncertainties on a daily basis and usually arrive at reasonable solutions. If it were otherwise, we would not be able to cope with the continually changing situations of our world.

In this and the following chapter, we shall discuss methods with which to accurately represent and deal with different forms of inconsistency, uncertainty, possibility, and beliefs. In other words, we shall be interested in representations and inference methods related to what is known as commonsense reasoning.

5.1 INTRODUCTION

Consider the following real-life situation. Timothy enjoys shopping in the mall only when the stores are not crowded. He has agreed to accompany Sue there on the following Friday evening since this is normally a time when few people shop. Before the given date, several of the larger stores announce a one-time, special sale starting on that Friday evening. Timothy, fearing large crowds, now retracts the offer to accompany Sue, promising to go on some future date. On the Thursday before the sale was to commence, weather forecasts predicted heavy snow. Now, believing the weather would discourage most shoppers, Timothy once again agreed to join Sue. But, unexpectedly, on the given Friday, the forecasts proved to be false; so Timothy once again declined to go.

This anecdote illustrates how one's beliefs can change in a dynamic environment. And, while one's beliefs may not fluctuate as much as Timothy's, in most situations, this form of belief revision is not uncommon. Indeed, it is common enough that we label it as a form of commonsense reasoning, that is, reasoning with uncertain knowledge.

Nonmonotonic Reasoning

The logics we studied in the previous chapter are known as monotonic logics. The conclusions derived using such logics are valid deductions, and they remain so. Adding new axioms increases the amount of knowledge contained in the knowledge base. Therefore, the set of facts and inferences in such systems can only grow larger; they can not be reduced; that is, they increase monotonically. The form of reasoning performed above by Timothy, on the other hand, is nonmonotonic. New facts became known which contradicted and invalidated old knowledge. The old knowledge was retracted causing other dependent knowledge to become invalid, thereby requiring further retractions. The retractions led to a shrinkage or nonmonotonic growth in the knowledge at times.

More formally, let KBL be a formal first order system consisting of a knowledge base and some logic L. Then, if KB1 and KB2 are knowledge bases where

KB1 = KBL
KB2 = KBL ∪ F, for some wff F, then
KB1 ⊆ KB2

In other words, a first order KB system can only grow monotonically with added knowledge.

When building knowledge-based systems, it is not reasonable to expect that all the knowledge needed for a set of tasks could be acquired, validated, and loaded into the system at the outset. More typically, the initial knowledge will be incomplete, contain redundancies, inconsistencies, and other sources of uncertainty. Even if it were possible to assemble complete, valid knowledge initially, it probably would not remain valid forever, not in a continually changing environment.

In an attempt to model real-world, commonsense reasoning, researchers have proposed extensions and alternatives to traditional logics such as PL and FOPL. The extensions accommodate different forms of uncertainty and nonmonotony. In some cases, the proposed methods have been implemented. In other cases they are still topics of research. In this and the following chapter, we will examine some of the more important of these methods.

We begin in the next section with a description of truth maintenance systems (TMS), systems which have been implemented to permit a form of nonmonotonic reasoning by permitting the addition of changing (even contradictory) statements to a knowledge base. This is followed in Section 5.3 by a description of other methods which accommodate nonmonotonic reasoning through default assumptions for incomplete knowledge bases. The assumptions are plausible most of the time, but may have to be retracted if other conflicting facts are learned. Methods to constrain the knowledge that must be considered for a given problem are considered next. These methods also relate to nonmonotonic reasoning. Section 5.4 gives a brief treatment of modal and temporal logics which extend the expressive power of classical logics to permit representations and reasoning about necessary and possible situations, temporal, and other related situations. Section 5.5 concludes the chapter with a brief presentation of a relatively new method for dealing with vague and imprecise information, namely fuzzy logic and language computation.

5.2 TRUTH MAINTENANCE SYSTEMS

Truth maintenance systems (also known as belief revision and revision maintenance systems), are companion components to inference systems. The main job of the TMS is to maintain consistency of the knowledge being used by the problem solver and not to perform any inference functions. As such, it frees the problem solver from any concerns of consistency and allows it to concentrate on the problem solution

aspects. The TMS also gives the inference component the latitude to perform nonmonotonic inferences. When new discoveries are made, this more recent information can displace previous conclusions that are no longer valid. In this way, the set of beliefs available to the problem solver will continue to be current and consistent.

Figure 5.1 illustrates the role played by the TMS as part of the problem solver. The inference engine (IE) solves domain problems based on its current belief set, while the TMS maintains the currently active belief set. The updating process is incremental. After each inference, information is exchanged between the two components. The IE tells the TMS what deductions it has made. The TMS, in turn, asks questions about current beliefs and reasons for failures. It maintains a consistent set of beliefs for the IE to work with even if new knowledge is added and removed.

For example, suppose the knowledge base (KB) contained only the propositions P, $P \rightarrow Q$, and modus ponens. From this, the IE would rightfully conclude Q and add this conclusion to the KB. Later, if it was learned that $\neg P$ was appropriate, it would be added to the KB resulting in a contradiction. Consequently, it would be necessary to remove P to eliminate the inconsistency. But, with P now removed, Q is no longer a justified belief. It too should be removed. This type of belief revision is the job of the TMS.

Actually, the TMS does not discard conclusions like Q as suggested. That could be wasteful, since P may again become valid, which would require that Q and facts justified by Q be rederived. Instead, the TMS maintains dependency records for all such conclusions. These records determine which set of beliefs are current (which are to be used by the IE). Thus, Q would be removed from the current belief set by making appropriate updates to the records and not by erasing Q. Since Q would not be lost, its rederivation would not be necessary if P became valid once again.

The TMS maintains complete records of reasons or justifications for beliefs. Each proposition or statement having at least one valid justification is made a part of the current belief set. Statements lacking acceptable justifications are excluded from this set. When a contradiction is discovered, the statements responsible for the contradiction are identified and an appropriate one is retracted. This in turn may result in other retractions and additions. The procedure used to perform this process is called dependency-directed backtracking. This process is described later.

The TMS maintains records to reflect retractions and additions so that the IE

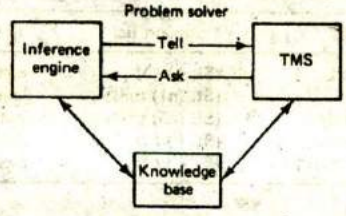

Figure 5.1 Architecture of the problem solver with a TMS.

will always know its current belief set. The records are maintained in the form of a dependency network. The nodes in the network represent KB entries such as premises, conclusions, inference rules, and the like. Attached to the nodes are justifications which represent the inference steps from which the node was derived. Nodes in the belief set must have valid justifications. A premise is a fundamental belief which is assumed to be always true. Premises need no justifications. They form a base from which all other currently active nodes can be explained in terms of valid justifications.

There are two types of justification records maintained for nodes: support lists (SL) and conceptual dependencies (CP). SLs are the most common type. They provide the supporting justifications for nodes. The data structure used for the SL contains two lists of other dependent node names, an in-list and an out-list. It has the form

(SL <in-list> <out-list>)

In order for a node to be active and, hence, labeled as IN the belief set, its SL must have at least one valid node in its in-list, and all nodes named in its out-list, if any, must be marked OUT of the belief set. For example, a current belief set that represents Cybil as a nonflying bird (an ostrich) might have the nodes and justifications listed in Table 5.1.

Each IN-node given in Table 5.1 is part of the current belief set. Nodes n1 and n5 are premises. They have empty support lists since they do not require justifications. Node n2, the belief that Cybil *can* fly is out because n3, a valid node, is in the out-list of n2.

Suppose it is discovered that Cybil is not an ostrich, thereby causing n5 to be retracted (marking its status as OUT). Then n3, which depends on n5, must also be retracted. This, in turn, changes the status of n2 to be a justified node. The resultant belief set is now that the bird Cybil can fly.

To represent a belief network, the symbol conventions shown in Figure 5.2 are sometimes used. The meanings of the nodes shown in the figure are (1) a premise is a true propositon requiring no justification, (2) an assumption is a current belief that could change, (3) a datum is either a currently assumed or IE derived belief, and (4) justifications are the belief (node) supports, consisting of supporting antecedent node links and a consequent node link.

TABLE 5.1 EXAMPLE NODES IN A DEPENDENCY NETWORK

Node	Status	Meaning	Support list	Comments
n1	IN	Cybil is a bird	(SL ( ) ( ))	a premise
n2	OUT	Cybil can fly	(SL (n1) (n3))	unjustified belief
n3	IN	Cybil cannot fly	(SL (n5) (n4))	justified belief
n4	OUT	Cybil has wings	(SL ( ) ( ))	retracted premise
n5	IN	Cybil is an Ostrich	(SL ( ) ( ))	a premise

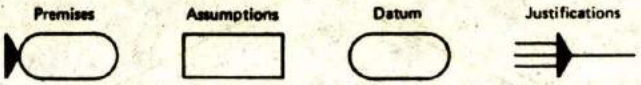

Figure 5.2 Belief network node meanings.

An example of a typical network representation is given in Figure 5.3. Note that nodes T, U, and W are OUT since they lack needed support from P. If the node labeled P is made IN for some reason, the TMS would update the network by propagating the "inness" support provided by node P to make T, U, and W IN.

As noted earlier, when a contradiction is discovered, the TMS locates the source of the contradiction and corrects it by retracting one of the contributing sources. It does this by checking the support lists of the contradictory node and going directly to the source of the contradiction. It goes directly to the source by examining the dependency structure supporting the justification and determining the offending nodes. This is in contrast to the naive backtracking approach which would search a deduction tree sequentially, node-by-node until the contradictory node is reached. Backtracking directly to the node causing the contradiction is known as dependency-directed backtracking (DDB). This is clearly a more efficient search strategy than chronological backtracking. This process is illustrated in Figure 5.4 where it is assumed that A and D are contradictory. By backtracking directly to

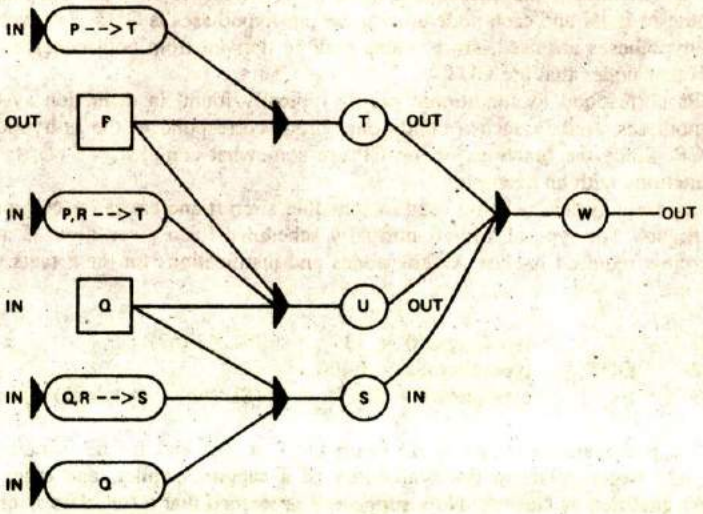

Figure 5.3 Typical fragment of a belief network.

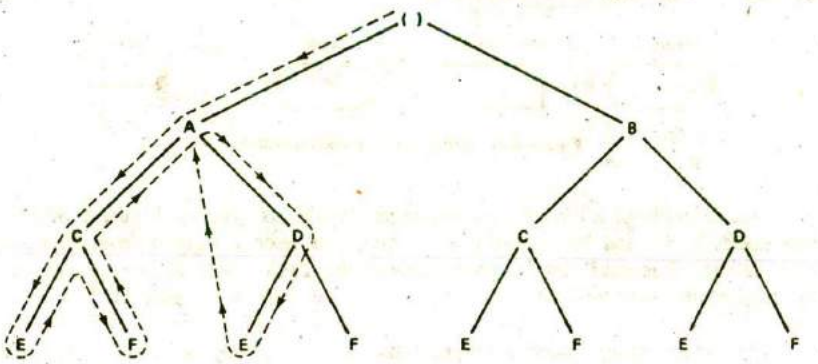

Figure 5.4 Dependency-directed backtracking in a TMS.

the source of a contradiction (the dashed line from E to A), extra search time is avoided.

CP justifications are used less frequently than the SLs. They justify a node as a type of valid hypothetical argument. The internal form of a CP justification is as follows:

(CP <consequent><inhypotheses><outhypotheses>)

A CP is valid if the consequent node is IN whenever each node among the in-hypotheses is IN and each node among the out-hypotheses is OUT. Two separate lists of hypotheses are used, since nodes may be derived from both nodes that are IN and other nodes that are OUT.

CPs correspond to conditional proofs typically found in deduction systems. The hypotheses used in such a conditional proof correspond to the in-hypotheses in the CP. Since the functions of the CP are somewhat complex, we clarify their main functions with an example.

Suppose a system is being used to schedule aircraft and crews for commercial airline flights. The type of aircraft normally scheduled for a given flight is a 737, and the crew required is class A. The nodes and justifications for these facts might be

n1	IN	type(aircraft) = 737	(SL ( ) (n2))
n2	OUT	type(aircraft) = L400	
n3	IN	class(crew) = A	(SL (n8, . . . , n22) ( ))

where the justifications for n1 is n2 (with OUT status) and for n3 is nodes n8, . . . , n22 which relate to the availability of a captain, copilot, and other crew members qualified as class A. Now suppose it is learned that a full class A crew is not available. To complete a schedule for the flight, the system must choose an

alternative aircraft, say an L400. To do this the IE changes the status of n2 to IN. But this results in a contradiction and, hence, creation of the node

```
n4    IN    contradiction    (SL (n1,n3) ( ))
```

The contradiction now initiates the DDB procedure in the TMS to locate the offending assumptions. Since there is only one such node, n1, its retraction is straightforward. For this, the TMS creates a "nogood" node with a CP justification as

```
n5    IN    nogood n1    (CP n4 (n1,n3) ( ))
```

To correct the inconsistency, the TMS next makes n2 an IN node by justifying it with n5 as follows

```
n2    IN    type(aircraft) = L400    (SL (n5) ( ))
```

This in turn causes n1 to become OUT (as an assumption n1 has a nonempty out-list). Also, since n4 was justified by n1, it too must become OUT. This gives the following set of nodes

```
n1    OUT   type(aircraft) = 737     (SL ( ) (n2))
n2    IN    type(aircraft) = L400    (SL (n5) ( ))
n3    IN    class(crew) = A          (SL (n8, . . . , n22) ( ))
n4    OUT   contradiction            (SL (n1,n3) ( ))
n5    IN    nogood n1                (CP n4 (n1,n3) ( ))
```

Note that a CP justification was needed for the "nogood" node to prevent a circular retraction of n2 from occurring. Had an SL been used for n5 with an n4 node in-list justification, n5 would have become OUT after n4, again causing n2 to become OUT.

The procedures for manipulating CPs are quite complicated, and, since they are usually converted into SLs anyway, we only mention their main functions here. For a more detailed account see Doyle (1979).

We have briefly described the JTMS here since it is the simplest and most widely used truth maintenance system. This type of TMS is also known as a nonmonotonic TMS (NMTMS). Several other types have been developed to correct some of the deficiencies of the JTMS and to meet other requirements. They include the logic-based TMS (the LTMS), and the assumption-based TMS (the ATMS), as well as others (de Kleer, 1986a and 1986b).

5.3 DEFAULT REASONING AND THE CLOSED WORLD ASSUMPTION

Another form of uncertainty occurs as a result of incomplete knowledge. One way humans deal with this problem is by making plausible default assumptions; that is, we make assumptions which typically hold but may have to be retracted if new

information is obtained to the contrary. For example, if you have an acquaintance named Pat who is over 20 years old, you would normally assume that this person would drive a car. Later, if you learned Pat had suffered from blackouts until just recently, you would be forced to revise your beliefs.

Default Reasoning

Default reasoning is another form of nonmonotonic reasoning; it eliminates the need to explicitly store all facts regarding a situation. Reiter (1980) develops a theory of default reasoning within the context of traditional logics. A default is expressed as

$$\frac{a(x) : Mb_1(x), \ldots, Mb_k(x)}{c(x)} \tag{5.1}$$

where $a(x)$ is a precondition wff for the conclusion wff $c(x)$, M is a consistency operator and the $b_i(x)$ are conditions, each of which must be separately consistent with the KB for the conclusion $c(x)$ to hold. As an example, suppose we wish to make the statement, "If x is an adult and it is consistent to assume that x can drive, then infer that x can drive." Using the above formula this would be represented as

$$\frac{\text{ADULT}(x) : \text{MDRIVE}(x)}{\text{DRIVE}(x)}$$

Default theories consist of a set of axioms and set of default inference rules with schemata like formula 5.1. The theorems derivable from a default system are those that follow from first-order logic and the assumptions assumed from the default rules.

Suppose a KB contains only the statements

$$\frac{\text{BIRD}(x) : \text{MFLY}(x)}{\text{FLY}(x)}$$

$$\text{BIRD(tweety)}$$

A default proof of FLY(tweety) is possible. But if KB also contains the clauses

$$\text{OSTRICH(tweety)}$$

$$\text{OSTRICH}(x) \rightarrow \neg\text{FLY}(x)$$

FLY(tweety) would be blocked since the default is now inconsistent.

Default rules are especially useful in hierarchial KBs. Because the default rules are transitive, property inheritance becomes possible. For example, in a heirarchy of living things, any animal could inherit the property has-heart from the rule

$$\forall x \ \text{ANIMAL}(x) \rightarrow \text{HAS-HEART}(x)$$

Transitivity can also be a problem in KBs with many default rules. Rule interactions can make representations very complex. Therefore caution is needed in implementing such systems.

Closed World Assumption

Another form of assumption, made with regard to incomplete knowledge, is more global in nature than single defaults. This type of assumption is useful in applications where most of the facts are known, and it is, therefore, reasonable to assume that if a proposition cannot be proven, it is false. This is known as the closed world assumption (CWA) with failure as negation (failure to prove a statement F results in assuming its negation, $\neg F$). This means that in a KB if the ground literal $P(a)$ is not provable, then $\neg P(a)$ is assumed to hold.

A classic example where this type of assumption is reasonable is in an airline KB application where city-to-city flights, not explicitly entered or provable, are assumed not to exist. Thus, ¬CONNECT(boston,van-horn) would be inferred whenever CONNECT(boston,van-horn) could not be derived from the KB. This seems reasonable since we would not want to enter all pairs of cities which do not have intercity flights.

By augmenting a KB with an assumption (a metarule) which states that if the ground atom $P(a)$ cannot be proved, assume its negation $\neg P(a)$, the CWA completes the theory with respect to KB. (Recall that a formal KB system is complete if and only if every ground atom or its negation is in the system.) Augmenting a KB with the negation of all ground atoms of the language which are not derivable, gives us a complete theory. For example, a KB containing only the clauses

$$P(a)$$

$$P(b)$$

$$P(a) \rightarrow Q(a)$$

and modus ponens is not complete, since neither $Q(b)$ nor $\neg Q(b)$ is inferrable from the KB. This KB can be completed, however, by adding either $Q(b)$ or $\neg Q(b)$.

In general, a KB augmented with CWA is not consistent. This is easily seen by considering the KB consisting of the clause

$$P(a) \vee Q(b)$$

Now, since none of the ground literals in this clause are derivable, the augmented KB becomes

$$P(a) \vee Q(b)$$

$$\neg P(a), \neg Q(b)$$

which is inconsistent.

It can be shown that consistency can be maintained for a special type of

CWA. This is a KB consisting of Horn clauses. If a KB is consistent and Horn, then its CWA augmentation is consistent. (Recall that Horn clauses are clauses with at most one positive literal.)

It may appear that the global nature of the negation as failure assumption is a serious drawback of CWA. And indeed there are many applications where it is not appropriate. There is a way around this using completion formulas which we discuss in the next section. Even so, CWA is essentially the formalism under which Prolog operates and Prolog has been shown to be effective in numerous applications.

5.4 PREDICATE COMPLETION AND CIRCUMSCRIPTION

Limiting default assumptions to only portions of a KB can be achieved through the use of completion or circumscription formulas. Unlike CWA, these formulas apply only to specified predicates, and not globally to the whole KB.

Completion Formulas

Completion formulas are axioms which are added to a KB to restrict the applicability of specific predicates. If it is known that only certain objects should satisfy given predicates, formulas which make this knowledge explicit (complete) are added to the KB. This technique also requires the addition of the unique-names assumption (UNA); that is, formulas which state that distinguished named entities in the KB are unique (different).

As an example of predicate completion, suppose we have the following KB:

OWNS(joe,ford)
STUDENT(joe)
OWNS(jill,chevy)
STUDENT(jill)
OWNS(sam,bike)
PROGRAMMER(sam)
STUDENT(mary)

If it is known that Joe is the only person who owns a Ford, this fact can be made explicit with the following completion formula:

$$\forall x \; \text{OWNS}(x,\text{ford}) \rightarrow \text{EQUAL}(x,\text{joe}) \tag{5.2}$$

In addition, we add the inequality formula

$$\neg\text{EQUAL}(a,\text{joe}) \tag{5.3}$$

which has the meaning that this is true for all constants a which are different from Joe.

Likewise, if it is known that Mary also has a Ford, and only Mary and Joe have Fords, the completion and corresponding inequality formulas in this case would be

$$\forall x\ \text{OWNS}(\text{ford},x) \rightarrow \text{EQUAL}(x,\text{joe}) \vee \text{EQUAL}(x,\text{mary})$$
$$\neg\text{EQUAL}(a,\text{joe})$$
$$\neg\text{EQUAL}(a,\text{mary})$$

Once completion formulas have been added to a KB, ordinary first order proof methods can be used to prove statements such as ¬OWNS(jill,ford). For example, to obtain a refutation proof using resolution, we put the completion and inequality formulas 5.2 and 5.3 in clausal form respectively, negate the query ¬OWNS(jill,ford) and add it to the KB. Thus, resolving with the following clauses

1. ¬OWNS(x,ford) V EQUAL(x,joe)
2. ¬EQUAL(a,joe)
3. OWNS(jill,ford)

from 1 and 3 we obtain

4. EQUAL(jill,joe)

and from 2 and 4 (recall that a is any constant not = Joe) we obtain the empty clause [ ], proving the query.

The need for the inequality formulas should be clearer now. They are needed to complete the proof by restricting the objects which satisfy the completed predicates.

Predicate completion performs the same function as CWA but with respect to the completed predicates only. However with predicate completion, it is possible to default both negative as well as positive statements.

Circumscription

Circumscription is another form of default reasoning introduced by John McCarthy (1980). It is similar to predicate completion in that all objects that can be shown to have some property P are in fact the only objects that satisfy P. We can use circumscription, like predicate completion, to constrain the objects which must be considered in any given situation. Suppose we have a university world situation in which there are two known CS students. We wish to state that the known students are the only students, to "circumscribe" the students

$$\text{CSSTUDENT}(a)$$
$$\text{CSSTUDENT}(b)$$

Let x be a tuple, that is $x = (x_1, \ldots, x_n)$, and let ϕ denote a relation of the same arity as P. Also let $\text{KB}(\phi(x))$ denote a KB with every occurrence of P

in KB replaced by ϕ. The usual form of circumscribing P in KB, denoted as CIR(KB:P), is given by

$$CIR(KB:P) = KB \ \& \ [\forall \phi (KB(\phi) \ \& \ (\forall x \phi(x) \rightarrow P(x))] \rightarrow \forall x [P(x) \rightarrow \phi(x)]$$

This amounts to replacing KB with a different KB in which some expression ϕ (in this example $\phi = (x = a \vee x = b)$) replaces each occurrence of P (here CSSTUDENT). The first conjunct inside the bracket identifies the required substitution. The second conjunct, $\forall x \phi(x) \rightarrow P(x)$, states that objects which satisfy ϕ also satisy P, while the conclusion states that ϕ and P are then equivalent.

Making the designated substitution in our example we obtain the circumscriptive inference

From CSSTUDENT(a) & CSSTUDENT(b), infer

$$\forall x (\text{CSSTUDENT}(x) \rightarrow (x = a \vee x = b)$$

Note that ϕ has been quantified in the circumscription schema above, making it a second order formula. This need not be of too much concern since in many cases the formula can be rewritten in equivalent first order form.

An interesting example used by McCarthy in motivating circumscription, related to the task of getting across a river when only a row boat was available. For problems such as this, it should only be necessary to consider those objects named as being important to the task completion and not innumerable other contingencies, like a hole in the boat, lost oars, breaking up on a rock, or building a bridge.

5.5 MODAL AND TEMPORAL LOGICS

Modal logics were invented to extend the expressive power of traditional logics. The original intent was to add the ability to express the necessity and possibility of propositions P in PL and FOPL. Later, other modal logics were introduced to help capture and represent additional subjective mood concepts (supposition, desire) in addition to the standard indicative mood (factual) concept representations given by PL and FOPL.

With modal logics we can also represent possible worlds in addition to the actual world in which we live. Thus, unlike traditional logics, where an interpretation of a wff results in an assignment of true or false (in one world only), an interpretation in modal logics would be given for each possible world. Consequently, the wff may be true in some worlds and false in others.

Modal logics are derived as extensions of PL and FOPL by adding modal operators and axioms to express the meanings and relations for concepts such as consistency, possibility, necessity, obligation, belief, known truths, and temporal situations, like past, present, and future. The operators take predicates as arguments and are denoted by symbols or letters such as L (it is necessary that), M (it is possible that), and so on. For example, MCOLDER-THAN(denver,portland) would be used to represent the statement "it is possible that Denver is colder than Portland."

Modal logics are classified by the type of modality they express. For example, alethic logics are concerned with necessity and possibility, deontic logics with what is obligatory or permissible, epistemic logics with belief and (known) knowledge, and temporal logics with tense modifiers like sometimes, always, what has been, what will be, or what is. In what follows, we shall be primarily concerned with alethic and temporal logics.

It is convenient to refer to an *agent* as the conceptualization of our knowledge-based system (a robot or some other KB system). We may adopt different views regarding the world or environment in which the agent functions. For example, one view may regard an agent as having a set of basic beliefs which consists of all statements that are derivable from the knowlege base. This was essentially the view taken in PL and FOPL. Note, however, that the statements we now call beliefs are not the same as known factual knowledge of the previous chapter. They may, in fact be erroneous.

In another view, we may treat the agent as having a belief set that is determined by possible worlds which are accessible to the agent. In what follows, we will adopt this latter view. But before describing the modal language of our agent, we should explain further the notion of possible worlds.

Possible Worlds

In different knowledge domains it is often productive to consider possible situations or events as alternatives to actual ones. This type of reasoning is especially useful in fields where an alternative course of action can lead to a significant improvement or to a catastrophic outcome. It is a common form of reasoning in areas like law, economics, politics, and military planning to name a few. Indeed, we frequently engage our imaginations to simulate possible situations which predict outcomes based on different scenarios. Our language has even grown to accommodate hypothetical concepts with statements like "if this were possible," or "suppose we have the following situation." On occasion, we may also wish to think of possible worlds as corresponding to different distributed knowledge bases.

Next, we wish to establish a relationship between an agent A and A's possible worlds. For this, we make use of a binary *accessibility* relation R. R is used to define relative possibilities between worlds for agent A. Let $W = \{w_0, w_1, \ldots\}$ denote a set of possible worlds where w_0 refers to the actual world. Let the relation $R(A: w_i, w_j)$ be defined on W such that for any $w_i, w_j \in W$, w_j is accessible from w_i for agent A whenever R is satisfied. This means that a proposition P is true in w_i if and only if P is true in all worlds accessible from w_i.

Since R is a relation, it can be characterized with relational properties such as reflexivity, transitivity, symmetry, and equivalence. Thus for any $w_i, w_j, w_k \in W$, the following properties are defined.

Reflexive. R is reflexive if $(w_i, w_i) \in R$ for each $w_i \in W$. In other words, all worlds are possible with respect to themselves.

Transitive. R is transitive if when $(w_i,w_j) \in R$, and $(w_j,w_k) \in R$ then $(w_i,w_k) \in R$. If w_j is accessible from w_i and w_k is accessible from w_j, then w_k is accessible from w_i.

Symmetric. R is symmetric if when $(w_i,w_j) \in R$ then $(w_j,w_i) \in R$. If w_j is accessible from w_i, then w_i is accessible from w_j.

Equivalence. R is an equivalence relation if R is reflexive, transitive, and symmetric.

These properties can also be described pictorially as illustrated in Figure 5.5.

Modal Operators and Logics

As suggested above, different operators are used to obtain different modalities. We begin with the standard definitions of a traditional logic, say PL, and introduce appropriate operators and, as appropriate, introduce certain axioms. For example, to define one first order modal logic we designate as L_{LM}, the operators L and M noted above would be included. We also add a necessity axiom which states that if the proposition P is a logical axiom then infer LP (that is, if P is universally valid, it is necessary that P is true). We also define LP = $\neg M \neg P$ or it is necessary that P is true as equivalent to it is not possible that P is not true.

Different modal logics can be obtained from L_{LM} by adding different axioms. Some axioms can also be related to the accessibility relations. Thus, for example, we can say that

If R is reflexive, add LP $\rightarrow$ P. In other words, if it is necessary that P is true in w_i, then P is true in w_i.

If R is transitive, add LP $\rightarrow$ LLP. This states that if P is true in all w_j accessible from some w_i, then P is true in all w_k accessible from w_j.

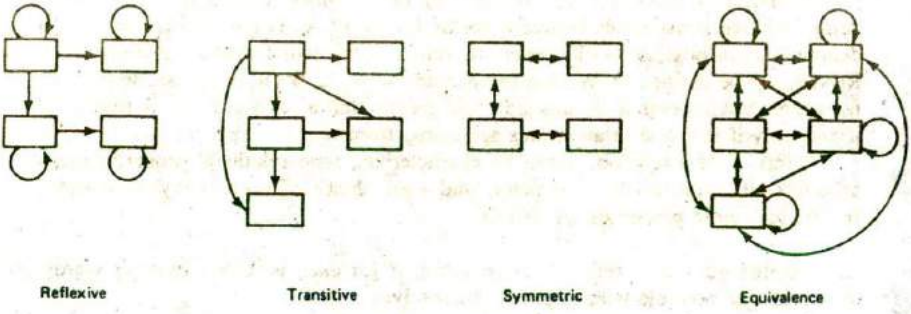

Figure 5.5 Accessibility relation properties.

If R is symmetric, then add P → LMP. This states that if P is true in w_i, then MP is true in all w_j accessible from w_i.

If R is an equivalence relation, add LP → P and MP → LMP.

To obtain a formal system from any of the above, it is necessary to add appropriate inference rules. Many of the inference rules defined in Chapter 4 would apply here as well, (modus ponens, universal substitution, and others). Thus, for a typical modal system based on propositional logic, the following axioms could apply:

A. The basic propositional logic assumptions (Chapter 4) including modus ponens,
B. Addition of the operators L (necessity) and M (possibility), and
C. Some typical modal axioms such as
   1. MP = ¬L¬P (possible P is equivalent to the statement it is not necessary that not P),
   2. L(P → Q) → (LP → LQ) (if it is necessary that P implies Q, then if P is necessary, Q is necessary),
   3. LP → P (if P is necessary then P), and
   4. LP → LLP (if P is necessary, then it is necessary that P is necessary).

Axioms C3 and C4 provide the reflexivity and transitivity access relations, respectively. If other relations are desired, appropriate axioms would be added to those given above; for example, for the symmetric access relation P → LMP would be added.

As an example of a proof in modal logic, suppose some assertions are added to a knowledge base for which the above assumptions and axioms apply:

D. Assertions:
   1. (sam is a man)
   2. M(sam is a child)
   3. L[(sam is a child) → L(sam is a child)]
   4. L[(sam is a man) → ¬(sam is a child)]

A simple proof that ¬(sam is a child) would proceed as follows. From C3 and D4 infer that

(sam is a man) → ¬(sam is a child)

From D1 and E1 using modus ponens conclude

¬(sam is a child)

Temporal Logics

Temporal logics use modal operators in relation to concepts of time, such as past, present, future, sometimes, always, preceeds, succeeds, and so on. An example of two operators which correspond to necessity and possibility are always (A) and

sometimes (S). A propositional temporal logic system using these operators would include the propositional logic assumptions of Chapter 4, the operators A and S, and appropriate axioms. Typical formulas using the A and S operators with the predicate Q would be written as

$AQ \rightarrow SQ$	(if always Q then sometimes Q)
$AQ \rightarrow Q$	(if always Q then Q)
$Q \rightarrow SQ$	(if Q then sometimes Q)
$SQ \rightarrow \neg A \neg Q$	(if sometimes Q then not always not Q)

A combination of propositional and temporal logic inference rules would then be added to obtain a formal system.

In addition to the above operators, the tense operators past (P) and future (F) have also been used. For example, tense formulas for the predicate (tweety files) would be written as

(tweety flies)	present tense.
P(tweety flies)	past tense: Tweety flew.
F(tweety flies)	future tense: Tweety will fly.
FP(tweety flies)	future perfect tense: Tweety will have flown.

It is also possible to define other modalities from the ones given above. For example, to express the concepts it has always been and it will always be the operators H and G may be defined respectively as

$HQ = \neg P \neg Q$	(it has always been the case that Q)
$GQ = \neg F \neg Q$	(it will always be the case that Q)

The four operators P, F, H, and G were used by the logician Lemmon (1965) to define a propositional tense logic he called KT. The language used consisted of propositional logic assumptions, the four tense operators P, F, H, and G, and the following axioms:

$$G(Q \rightarrow R) \rightarrow (GQ \rightarrow GR)$$
$$H(Q \rightarrow R) \rightarrow (HQ \rightarrow HR)$$
$$\neg G \neg HQ \rightarrow Q$$
$$\neg H \neg GQ \rightarrow Q$$

To complete the formal system, inference rules of propositional logic such as modus ponens and the two rules

$$\frac{Q}{HQ} \qquad \frac{Q}{GQ}$$

were added. These rules state that if Q is true, infer that Q has always been the case and Q will always be the case, respectively.

The semantics of a temporal logic is closely related to the frame problem. The frame problem is the problem of managing changes which occur from one situation to another or from frame to frame as in a moving picture sequence.

For example, a KB which describes a robot's world must know which facts change as a result of some action taken by the robot. If the robot throws out the cat, turns off the light, leaves the room, and locks the door, some, but not all facts which describe the room in the new situation must be changed.

Various schemes have been proposed for the management of temporally changing worlds including other modal operators and time and date tokens. This problem has taken on increased importance and a number of solutions have been offered. Still much work remains to find a comprehensive solution.

5.6 FUZZY LOGIC AND NATURAL LANGUAGE COMPUTATIONS

We have already noted several limitations of traditional logics in dealing with uncertain and incomplete knowledge. We have now considered methods which extend the expressive power of the traditional logics and permit different forms of nonmonotonic reasoning. All of the extensions considered thus far have been based on the truth functional features of traditional logics. They admit interpretations which are either true or false only. The use of two valued logics is considered by some practitioners as too limiting. They fail to effectively represent vague or fuzzy concepts.

For example, you no doubt would be willing to agree that the predicate "TALL" is true for Pole, the seven foot basketball player, and false for Smidge the midget. But, what value would you assign for Tom, who is 5 foot 10 inches? What about Bill who is 6 foot 2, or Joe who is 5 foot 5? If you agree 7 foot is tall, then is 6 foot 11 inches also tall? What about 6 foot 10 inches? If we continued this process of incrementally decreasing the height through a sequence of applications of modus ponens, we would eventually conclude that a three foot person is tall. Intuitively, we expect the inferences should have failed at some point, but at what point? In FOPL there is no direct way to represent this type of concept. Furthermore, it is not easy to represent vague statements such as "slightly more beautiful," "not quite as young as . . . ." "not very expensive, but of questionable reliability," "a little bit to the left," and so forth.

Theory of Fuzzy Sets

In standard set theory, an object is either a member of a set or it is not. There is no in between. The natural numbers 3, 11, 19, and 23 are members of the set of prime numbers, but 10, blue, cow, and house are not members. Traditional logics are based on the notions that P(a) is true as long as a is a member of the set belonging to class P and false otherwise. There is no partial containment. This

amounts to the use of a characteristic function f for a set A; where $f_A(x) = 1$ if x is in A; otherwise it is 0. Thus, f is defined on the universe U and for all $x \in U$, $f: U \rightarrow \{0,1\}$.

We may generalize the notion of a set by allowing characteristic functions to assume values other than 0 and 1. For example, we define the notion of a fuzzy set with the characteristic function u which maps from U to a number in the real interval $[0,1]$; that is $u: U \rightarrow [0,1]$. Thus, we define the fuzzy set $\tilde{A}$ as follows (the ~ symbol is omitted but assumed when a fuzzy set appears in a subscript):

Definition. Let U be a set, denumerable or not, and let x be an element of U. A fuzzy subset $\tilde{A}$ of U is a set of ordered pairs $\{(x,u_A(x))\}$, for all x in U, where $u_A(x)$ is a membership characteristic function with values in $[0,1]$, and which indicates the degree or level of membership of x in $\tilde{A}$.

A value of $u_A(x) = 0$ has the same meaning as $f_A(x) = 0$, that x is not a member of $\tilde{A}$, whereas a value of $u_A(x) = 1$ signifies that x is completely contained in $\tilde{A}$. Values of $0 < u_A(x) < 1$ signify that x is a partial member of $\tilde{A}$.

Characteristic functions for fuzzy sets should not be confused with probabilities. A probability is a measure of the degree of uncertainty, likelihood, or belief based on the frequency or proportion of occurrence of an event. Whereas a fuzzy characteristic function relates to vagueness and is a measure of the feasibility or ease of attainment of an event. Fuzzy sets have been related to possibility distributions which have some similarities to probability distributions, but their meanings are entirely different.

Given a definition of a fuzzy set, we now have a means of expressing the notion TALL(x) for an individual x. We might for example, define the fuzzy set $\tilde{A}$ = {tall} and assign values of $u_A(0) = u_A(10) = \ldots = u_A(40) = 0$, $u_A(50) = 0.2$, $u_A(60) = 0.4$, $u_A(70) = 0.6$, $u_A(80) = 0.9$, $u_A(90) = u_A(100) = 1.0$. Now we can assign values for individuals noted above as TALL(Pole) = 1.0, and TALL (Joe) = 0.5. Of course, our assignment of values for $u_A(x)$ was purely subjective. And to be complete, we should also assign values to other fuzzy sets associated with linguistic variables such as very short, short, medium, etc. We will discuss the notions of linguistic variables and related topics later.

Operations on fuzzy sets are somewhat similar to the operations of standard set theory. They are also intuitively acceptable.

$\tilde{A} = \tilde{B}$ if and only if $u_A(x) = u_B(x)$ for all $x \in U$	equality
$\tilde{A} \subseteq \tilde{B}$ if and only if $u_A(x) \leq u_B(x)$ for all $x \in U$	containment
$u_{A \cap B}(x) = \min_x\{u_A(x), u_B(x)\}$	intersection
$u_{A \cup B}(x) = \max_x\{u_A(x), u_B(x)\}$	union
$u_{A'}(x) = 1 - u_A(x)$	complement set

The single quotation mark denotes the complement fuzzy set, A'. Note that the intersection of two fuzzy sets $\tilde{A}$ and $\tilde{B}$ is the largest fuzzy subset that is a subset of

both. Likewise, the union of two fuzzy sets $\tilde{A}$ and $\tilde{B}$ is the smallest fuzzy subset having both $\tilde{A}$ and $\tilde{B}$ as subsets.

With the above definitions, it is possible to derive a number of properties which hold for fuzzy sets much the same as they do for standard sets. For example, we have

$$\tilde{A} \cup (\tilde{B} \cap \tilde{C}) = (\tilde{A} \cup \tilde{B}) \cap (\tilde{A} \cup \tilde{C}) \qquad \text{distributivity}$$

$$\tilde{A} \cap (\tilde{B} \cup \tilde{C}) = (\tilde{A} \cap \tilde{B}) \cup (\tilde{A} \cap \tilde{C})$$

$$(\tilde{A} \cup \tilde{B}) \cup \tilde{C} = \tilde{A} \cup (\tilde{B} \cup \tilde{C}) \qquad \text{associativity}$$

$$(\tilde{A} \cap \tilde{B}) \cap \tilde{C} = \tilde{A} \cap (\tilde{B} \cap \tilde{C})$$

$$\tilde{A} \cap \tilde{B} = \tilde{B} \cap \tilde{A}, \quad \tilde{A} \cup \tilde{B} = \tilde{B} \cup \tilde{A} \qquad \text{commutativity}$$

$$\tilde{A} \cap \tilde{A} = \tilde{A}, \quad \tilde{A} \cup \tilde{A} = \tilde{A} \qquad \text{idempotency}$$

There is also a form of DeMorgan's laws:

$$u_{(A \cap B)'}(x) = u_{A' \cup B'}(x)$$

$$u_{(A \cup B)'}(x) = u_{A' \cap B'}(x)$$

Note however, that

$$\tilde{A} \cap \tilde{A}' \neq \emptyset, \tilde{A} \cup \tilde{A}' \neq U$$

since in general for $u_A(x) = a$, with $0 < a < 1$, we have

$$u_{A \cup A'}(x) = \max[a, 1 - a] \neq 1$$

$$u_{A \cap A'}(x) = \min[a, 1 - a] \neq 0$$

On the other hand the following relations do hold:

$$\tilde{A} \cap \emptyset = \emptyset \qquad \tilde{A} \cup \emptyset = \tilde{A}$$

$$\tilde{A} \cap U = \tilde{A} \qquad \tilde{A} \cup U = U$$

The universe from which a fuzzy set is constructed may also be uncountable. For example, we can define values of u for the fuzzy set $\tilde{A}$ = {young} as

$$u_A(x) = \begin{cases} 1.0 & \text{for } 0 \leq x \leq 20 \\ \left[1 + \left(\dfrac{x - 20}{10}\right)^2\right]^{-1} & \text{for } x > 20 \end{cases}$$

The values of $u_A(x)$ are depicted graphically, in Figure 5.6.

A number of operations that are unique to fuzzy sets have been defined. A few of the more common operations include

Dilation. The dilation of $\tilde{A}$ is defined as

$$\text{DIL}(\tilde{A}) = [u_A(x)]^{1/2} \qquad \text{for all } x \text{ in } U$$

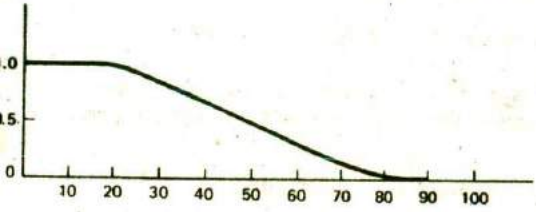

Figure 5.6 Degrees of membership for young age.

Concentration. The concentration of $\tilde{A}$ is defined as

$$\text{CON}(\tilde{A}) = [u_A(x)]^2 \qquad \text{for all } x \text{ in } U$$

Normalization. The normalization of $\tilde{A}$ is defined as

$$\text{NORM}(\tilde{A}) = u_A(x) \,/\, \max_x\{u_A(x)\} \qquad \text{for all } x \text{ in } U$$

These three operations are depicted graphically in Figure 5.7. Note that dilation tends to increase the degree of membership of all partial members x by spreading out the characteristic function curve. The concentration is the opposite of dilation. It tends to decrease the degree of membership of all partial members, and concentrates the characteristic function curve. Normalization provides a means of normalizing all characteristic functions to the same base much the same as vectors can be normalized to unit vectors.

In addition to the above, a number of other operations have been defined including fuzzification. Fuzzification permits one to fuzzify any normal set. These operations will not be required here. Consequently we omit their definitions.

Lotfi a. Zadeh of the University of California, Berkeley, first introduced fuzzy sets in 1965. His objectives were to generalize the notions of a set and propositions to accommodate the type of fuzziness or vagueness we have discussed above. Since its introduction in 1965, much research has been conducted in fuzzy set theory and logic. As a result, extensions now include such concepts as fuzzy arithmetic, possibility distributions, fuzzy statistics, fuzzy random variables, and fuzzy set functions.

Many researchers in AI have been reluctant to accept fuzzy logic as a viable alternative to FOPL. Still, successful systems which employ fuzzy logic have been

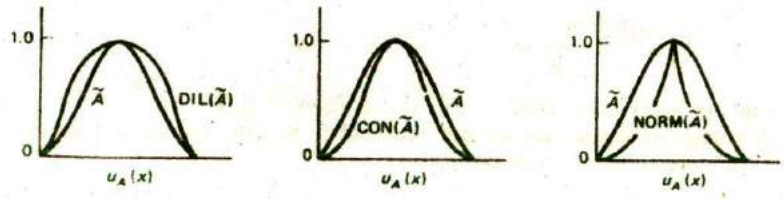

Figure 5.7 Dilation, concentration, and normalization of u.

developed, and a fuzzy VLSI chip has been produced by Bell Telephone Laboratories, Inc., of New Jersey (Togai and Watanabe, 1986).

Reasoning with Fuzzy Logic

The characteristic function for fuzzy sets provides a direct linkage to fuzzy logic. The degree of membership of x in $\tilde{A}$ corresponds to the truth value of the statement x is a member of $\tilde{A}$ where $\tilde{A}$ defines some propositional or predicate class. When $u_A(x) = 1$, the proposition A is completely true, and when $u_A(x) = 0$ it is completely false. Values between 0 and 1 assume corresponding values of truth or falsehood.

In Chapter 4 we found that truth tables were useful in determining the truth value of a statement or wff. In general this is not possible for fuzzy logic since there may be an infinite number of truth values. One could tabulate a limited number of truth values, say those corresponding to the terms false, not very false, not true, true, very true, and so on. More importantly, it would be useful to have an inference rule equivalent to a fuzzy modus ponens.

Generalized modus ponens for fuzzy sets have been proposed by a number of researchers. They differ from the standard modus ponens in that statements which are characterized by fuzzy sets are permitted and the conclusion need not be identical to the implicand in the implication. For example, let $\tilde{A}$, $\tilde{A}1$, $\tilde{B}$, and $\tilde{B}1$ be statements characterized by fuzzy sets. Then one form of the generalized modus ponens reads

Premise: x is $\tilde{A}1$

Implication: If x is $\tilde{A}$ then y is $\tilde{B}$

Conclusion: y is $\tilde{B}1$

An example of this form of modus ponens is given as

Premise: This banana is very yellow

Implication: If a banana is yellow then the banana is ripe

Conclusion: This banana is very ripe

Although different forms of fuzzy inference have been proposed, we present only Zadeh's original compositional rule of inference.

First, recall the definition of a relation. For two sets A and B, the Cartesian product $A \times B$ is the set of all ordered pairs (a,b), for $a \subseteq A$ and $b \subseteq B$. A binary relation on two sets A and B is a subset of $A \times B$. Likewise, we define a binary fuzzy relation $\tilde{R}$ as a subset of the fuzzy Cartesian product $\tilde{A} \times \tilde{B}$, a mapping of $\tilde{A} \rightarrow \tilde{B}$ characterized by the two parameter membership function $u_R(a,b)$. For example, let $\tilde{A} = \tilde{B} = \mathbb{R}$ the set of real numbers, and let $\tilde{R}$: = much larger than. A membership function for this relation might then be defined as

$$u_R(a,b) = \begin{cases} 0 & \text{for } a \leq b \\ (1 + (a-b)^{-2})^{-1} & \text{for } a > b \end{cases}$$

Now let X and Y be two universes and let $\tilde{A}$ and $\tilde{B}$ be fuzzy sets in X and $X \times Y$ respectively. Define fuzzy relations $\tilde{R}_A(x)$, $\tilde{R}_B(x,y)$, and $\tilde{R}_C(y)$ in X, $X \times Y$ and Y, respectively. Then the compositional rule of inference is the solution of the relational equation

$$\tilde{R}_C(y) = \tilde{R}_A(x) \circ \tilde{R}_B(x,y) = \max_x \min\{u_A(x), u_B(x,y)\}$$

where the symbol o signifies the composition of $\tilde{A}$ and $\tilde{B}$. As an example, let

$$X = Y = \{1,2,3,4\}$$

$$\tilde{A} = \{\text{little}\} = \{(1 / 1),(2 / .6),(3 / .2),(4 / 0)$$

$\tilde{R}$ = approximately equal, a fuzzy relation defined by

$\tilde{R}$: x \ y	1	2	3	4
1	1	.5	0	0
2	.5	1	.5	0
3	0	.5	1	.5
4	0	0	.5	1

Then applying the max-min composition rule

$$\begin{aligned}\tilde{R}_C(y) &= \max_x \min\{u_A(x), u_R(x,y)\} \\ &= \max_x \{\min[(1,1),(.6,.5),(.2,0),(0,0)], \\ &\qquad \min[(1,.5),(.6,1),(.2,.5),(0,0)], \\ &\qquad \min[(1,0),(.6,.5),(.2,1),(0,.5)], \\ &\qquad \min[(1,0),(.6,0),(.2,.5),(0,1)]\} \\ &= \max_x\{[1,.5,0,0],[.5,.6,.2,0],[0,.5,.2,0],[0,0,.2,0]\} \\ &= \{[1],[.6],[.5],[.2]\}\end{aligned}$$

Therefore the solution is

$$\tilde{R}_C(y) = \{(1 /1),(2 /.6),(3 /.5),(4/.2)\}.$$

Stated in terms of a fuzzy modus ponens, we might interpret this as the inference

Premise: x is little

Implication: x and y are approximately equal

Conclusion: y is more or less little

The above notions can be generalized to any number of universes by taking the Cartesian product and defining relations on various subsets.

Natural Language Computations

Earlier, we mentioned linguistic variables without really defining them. Linguistic variables provide a link between a natural or artifical language and representations which accommodate quantification such as fuzzy propositions. In this section, we

formally define a linguistic variable and show how they are related to fuzzy logic.

Informally, a linguistic variable is a variable that assumes a value consisting of words or sentences rather than numbers. For example, the variable AGE might assume values of very young, young, not young, or not old. These values, which are themselves linguistic variables, may in turn, each be given meaning through a base universe of elapsed time (years). This is accomplished through appropriately defined characteristic functions.

As an example, let the linguistic variable AGE have possible values {very young, young, not young, middle-aged, not old, old, very old}. To each of these variables, we associate a fuzzy set consisting of ordered tuples $\{(x/u_A(x))\}$ where $x \in U = [0,110]$ the universe (years). The variable AGE, its values, and their corresponding fuzzy sets are illustrated in Figure 5.8.

A formal, more elegant definition of a linguistic variable is one which is based on language theory concepts. For this, we define a linguistic variable as the quintuple

$$(x,T(x),U,G,M)$$

where

x is the name of the variable (AGE in the above example),

$T(x)$ is the terminal set of x (very-young, young, etc.),

U is the universe of discourse (years of life),

G is a set of syntactic rules, the grammer which generates the values of x, and

M is the semantic rule which associates each value of x with its meaning $M(x)$, a fuzzy subset of U.

The grammar G is further defined as the tuple (V_N,V_T,P,S) where V_N is the set of nonterminal symbols, V_T the set of terminal symbols from the alphabet of G, P is

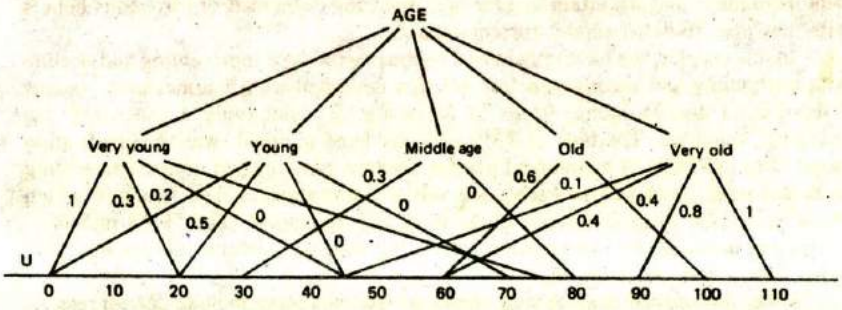

Figure 5.8 The linguistic variable *age*.

a set of rewrite (production) rules, and S is the start symbol. The language $L(G)$ generated by the grammar G is the set of all strings w derived from S consisting of symbols in V_T. Thus, for example, using $V_N = \{A,B,C,D,E,S\}$ and the following rules in P, we generate the terminal string "not young and not very old."

P:	$S \rightarrow A$	$A \rightarrow A$ and B	$C \rightarrow D$	$D \rightarrow$ young
	$S \rightarrow$ S or A	$B \rightarrow C$	$C \rightarrow$ very C	$E \rightarrow$ old
	$A \rightarrow B$	$B \rightarrow$ not C	$C \rightarrow E$	

This string is generated through application of the following rules:

$S \rightarrow A \rightarrow A$ and $B \rightarrow A$ and not $C \rightarrow A$ and not very $C \rightarrow A$ and not very $E \rightarrow A$ and not very old $\rightarrow B$ and not very old $\rightarrow$ not C and not very old $\rightarrow$ not D and not very old $\rightarrow$ not young and not very old

The semantic rule M gives meaning to the values of AGE. For example, we might have $M(\text{old}) = \{(x / u_{\text{old}}(x)) \mid x \in [0,110]\}$ where $u_{\text{old}}(x)$ is defined as

$$u_{\text{old}}(x) = \begin{cases} 0 & \text{for } 0 \leq x \leq 50 \\ \left(1 + \left(\frac{x - 50}{5}\right)^{-2}\right)^{-1} & \text{for } x > 50 \end{cases}$$

In this section we have been able to give only a brief overview of some of the concepts and issues related to the expanding fields based on fuzzy set theory. The interested reader will find numerous papers and texts available in the literature. We offer a few representative references here which are recent and have extensive bibliographies: (Zadeh, 1977, 1981, 1983), (Kandel, 1982), (Gupta et al., (1985), and (Zimmerman, 1985).

5.7 SUMMARY

Commonsense reasoning is nonmonotonic in nature. It requires the ability to reason with incomplete and uncertain knowledge, replacing outmoded or erroneous beliefs with new ones to better model current situations.

In this chapter, we have considered various methods of representing and dealing with uncertainty and inconsistencies. We first described truth maintenance systems which permit nonmonotonic forms of reasoning by maintaining a consistent, but changing, belief set. The type of TMS we considered in detail, was the justification based TMS or JTMS. It maintained a belief network consisting of nodes representing facts and rules. Attached to each node was a justification, a data structure which characterized the In or Out status of the node and its support. The JTMS maintains the current belief set for the problem solver, but does not perform inferencing functions. That is the job of the IE.

Other methods of dealing with nonmonotonic reasoning include default reasoning and CWA. In both cases, assumptions are made regarding knowledge or beliefs which are not directly provable. In CWA, if P can not be proved, then assume $\neg$P

is true. Default reasoning is based on the use of typical assumptions about an object or class of objects which are plausible. The assumptions are regarded as valid unless new contrary information is learned.

Predicate completion and circumscription are methods which restrict the values a predicate or group of predicates may assume. They allow the predicates to take on only those values the KB says they must assume. Both methods are based on the use of completion formulas. Like CWA, they are also a form of nonmonotonic reasoning.

Modal logics extend the expressiveness of classical logics by permitting the notions of possibility, necessity, obligation, belief, and the like. A number of different modal logics have been formalized, and inference rules comparable to propositional and predicate logic are available to permit different forms of nonmonotonic reasoning.

Like modal logics, fuzzy logic was introduced to generalize and extend the expressiveness of traditional logics. Fuzzy logic is based on fuzzy set theory which permits partial set membership. This, together with the ability to use linguistic variables, makes it possible to represent such notions as Sue is not very tall, but she is quite pretty.

EXERCISES

5.1 Give an example of nonmonotonic reasoning you have experienced at some time.

5.2 Draw the TMS belief network for the following knowledge base of facts. The question marks under output status means that the status must be determined for these datum nodes.

INPUTS

Premises	Status	Assumptions	Status
$Q \rightarrow S$	IN	Q	OUT
$Q, R \rightarrow U$	IN	R	IN
$P, R \rightarrow T$	IN		. . .
P	IN		
	. . .		

OUTPUTS

Datum	Status	Conditions
S	?	If Q, $Q \rightarrow S$ then S
U	?	If Q; $Q, R \rightarrow U$, R then U
T	?	If R; $P, R \rightarrow T$; P then T

5.3 Draw a TMS belief network for the aircraft example described in Section 5.2 and show how the network changes with the selection of an alternate choice of aircraft.

5.4 Write schemata for default reasoning using the following statements:

(a) If someone is an adult and it is consistent to assume that adults can vote, infer that that person can vote.